

The pathwidth complex

Alexandre Linhares

affiliation to be added

Horacio Hideki Yanasse

affiliation to be added

Draft, 3 October 2026

Abstract

The minimization of open stacks problem (MOSP) asks for the order in which to make a set of patterns so that the fewest customer orders are open at once. [Linhares and Yanasse \(2002\)](#) listed twelve problems from operations research, VLSI design and graph theory, MOSP among them, as equal to it “plus or minus one”, and gave no proofs. We ask which of these equalities are true, and what follows when results, algorithms and benchmarks are moved between the twelve problems through pathwidth, the quantity that connects them. We find four things. First, eight of the twelve rows are exact, counting pathwidth itself: each equals pathwidth or pathwidth plus one. Two lie in bands one and two wide, and two are false as stated; every relation is proved, and machine-checked in Lean. Second, the literature has settled on the name pathwidth, used more than three times as often as the other eleven names together, and the three communities rarely cite one another. Third, the best exact algorithm for MOSP, the search of [Chu and Stuckey \(2009\)](#), is a pathwidth algorithm, and two of its published dominance theorems are false. The correct rule was already known for pathwidth; we show that it is tested by one bipartite matching per candidate, and prove the repaired search sound. In our implementation it costs at most 2% more search nodes on each benchmark set, and it re-proved all 115 benchmark optima that rested on the published rules, changing none. Fourth, we release 17,714 benchmark graphs, distinct up to isomorphism, from twenty collections, 16,087 of them with a certified pathwidth and a layout that attains it.

1 Introduction

A factory makes a set of patterns, one at a time, and each customer’s order needs some of them. A customer’s stack opens when the first pattern the customer needs is made, and it closes when the last one is made. The *minimization of open stacks problem* (MOSP) asks for the order of the patterns that keeps the fewest stacks open at any one time. In the cutting industry, where the problem arose, the patterns are cutting patterns and the customers are piece types ([Yanasse, 1997b](#)). Figure 1.1(a) gives a small instance. Six customers, a to f , need four patterns: $P_1 = \{a, b\}$, $P_2 = \{b, c, d\}$, $P_3 = \{c, d, e\}$ and $P_4 = \{e, f\}$. Made in order A, $P_1P_2P_3P_4$, the patterns keep at most 3 stacks open. Made in order B, $P_1P_3P_4P_2$, they keep 5 open, because customer b is served first but waits for P_2 , which comes last. Of the 24 orders, 2 reach 3 (order A and its reverse), 16 give 4 and 6 give 5. MOSP is NP-hard ([Linhares, 2001](#); [Linhares and Yanasse, 2002](#)).

The same matrix is a problem in VLSI design. In a *gate matrix layout* the patterns are gates, drawn as vertical lines in some order, and the customers are nets. Each net is a horizontal wire from the first gate it connects to the last, and wires that overlap must lie on different horizontal tracks. The task is to order the gates so that the fewest tracks suffice. Figure 1.1(b) and (c) draw the two orders. The wires are the same; only the gate order changes which wires overlap. Order A packs into 3 tracks and order B needs 5. The number of tracks a gate order needs is its *density*, the most wires crossing any one gate, and that is exactly the number of open stacks there. So the two problems compute the same number from the same matrix ([Linhares and Yanasse, 2002](#), Proposition 2).

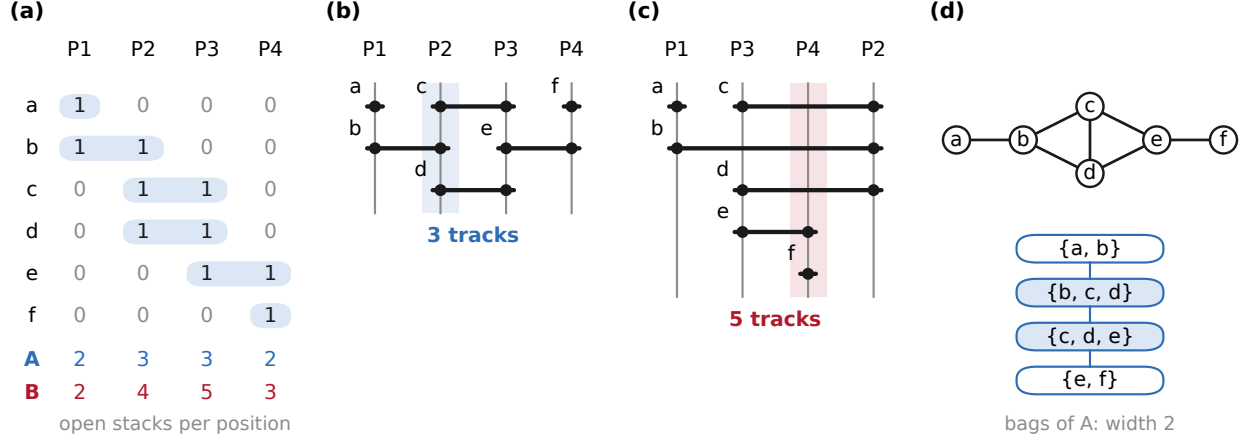


Figure 1.1: One instance, three readings, one number. (a) Six customers and four patterns, with each customer’s open stack shaded in order A; order A keeps at most 3 stacks open and order B at most 5, and only 2 of the 24 orders reach 3. (b, c) The same matrix as a gate matrix layout: the same two orders need 3 and 5 tracks (the shaded gate is where the density peaks). (d) The same matrix as a graph, one clique per pattern: the open sets of order A are the bags of a path decomposition of width 2, so the optimum is the pathwidth plus one (Theorem 2.3).

The same matrix is also a graph. Make each customer a vertex, and join two customers when some pattern needs both, so that each pattern becomes a clique (Figure 1.1(d)). Write down, for order A, the set of open stacks at each position: $\{a, b\}$, $\{b, c, d\}$, $\{c, d, e\}$, $\{e, f\}$. These sets cover every vertex and every edge, and each customer appears in a run of consecutive sets. In graph theory such a sequence is a *path decomposition*, its sets are *bags*, and its width is the size of its largest bag minus one. The least width over all path decompositions is the *pathwidth* of the graph, here 2. The construction works in both directions, so the fewest open stacks is always the pathwidth plus one (Theorem 2.3).

Linhaires and Yanasse (2002) collected twelve such problems, from operations research, VLSI design and graph theory, in the table that Table 1.1 reproduces. Each computes, on corresponding inputs, “a function $f(\Pi)$ that is either equal to the number of open stacks or closely related to it (plus or minus one)” (Linhaires and Yanasse, 2002, p. 1764). The table gives a reference for each problem, but no proofs and no graph of which problem reduces to which. “Plus or minus one” has two readings. Under the *literal* reading, f is within one of the number of open stacks on every input. Under the *fixed-offset* reading, f equals the number of open stacks plus one constant, the same on every input. Only the second makes solving one problem the same as solving another. We call the twelve *the pathwidth complex*, because pathwidth is both the name the literature uses most (Section 3) and the quantity through which every true relation in the table is proved (Section 2). This paper asks: *which of the equalities in the Linhares–Yanasse table are true, and what follows when results, algorithms and benchmarks are moved between the twelve problems through pathwidth?*

The table has a history that explains why the question is still open. It was first printed as Table 2.1 of the first author’s doctoral thesis (Linhaires, 2001, p. 13), supervised by the second author, which proved MOSP NP-hard and equal to gate matrix layout. Its rows were collected over the years of the doctorate, while reading the literature of each field. Each came with a reference and nothing else. No row’s relation to open stacks was checked, no published proof was restated, and no graph said which problem reduces to which. The table appeared again, row for row, as Table 1 of Linhares and Yanasse (2002), and it has been read since as a statement of equivalence. Garcia de la Banda and Stuckey (2007), for instance, credit the speed of their search in this journal to “the fact that the problem is fixed parameter tractable (Linhaires and Yanasse 2002)”. So the table is already used to justify algorithms, and whether it is true matters. If it is true, every result about any one of the twelve problems transfers to all the others. Looking back, the table was the thesis’s most important finding, and this article does what the thesis did not: it states each problem precisely, proves

Table 1.1: The Linhares–Yanasse table, reproduced from Linhares and Yanasse (2002, Table 1, p. 1764) with its row order, wording and references as printed, and its caption: “Display of a family of equivalent problems studied independently on the literature”. The shaded column is ours. It gives the relation to pathwidth (pw) that is true, proved in Section 2 (Table 2.2). Eight rows are exact, counting the pathwidth row itself; two are bands; two are false as the table states them.

Problem	Discipline	References	This paper
MOSP	Operations research	Yanasse (1997b); Fink and Voß (1999)	exact: $\text{pw} + 1$
Gate matrix layout	VLSI design	Möhring (1990); Wing et al. (1985)	exact: $\text{pw} + 1$
One-dimensional logic	VLSI design	Ohtsuki et al. (1979)	exact: $\text{pw} + 1$
PLA folding	VLSI design	Möhring (1990)	false ^a
Interval thickness	Graph theory	Kashiwabara and Fujisawa (1979)	exact: $\text{pw} + 1$
Node search game	Graph theory	Kirousis and Papadimitriou (1985)	exact: $\text{pw} + 1$
Edge search game	Graph theory	Kirousis and Papadimitriou (1986)	band: pw to $\text{pw} + 2$
Narrowness	Graph theory	Kornai and Tuza (1992)	exact: $\text{pw} + 1$
Split bandwidth	Graph theory	Fomin (1998)	band: pw to $\text{pw} + 1$
Graph path-width	Graph theory	Kinnersley (1992)	exact: pw (reference)
Edge separation	Graph theory	Lengauer (1981)	false ^b
Vertex separation	Graph theory	Kinnersley (1992)	exact: pw

^aExact ($\text{pw} + 1$) for multiple folding, with any number of nets per track. ^bAs cutwidth; the vertex separator game of the same reference is exact (pw , for graphs with an edge).

what is true and refutes what is not. The communities that could have checked the table rarely read each other: of 844 works that cite the table’s papers, only six cite both an open-stacks paper and a graph-theory paper (Section 3).

What we find.

- *The table, settled.* Eight of the twelve rows are exact, the pathwidth row itself counted as one of them: the other seven equal pathwidth or pathwidth plus one on every input, outside degenerate cases such as an empty graph. Two rows, split bandwidth and edge search, lie in bands one and two wide. Two rows, PLA folding and edge separation, are false as stated, with counterexamples. A variant of each false row is exact (Section 2).
- *One name.* The literature has settled on “pathwidth”, which has more than three times as many relevant works as the other eleven names together. Besides pathwidth, the most active community over 2005–2024 is the open-stacks one, and the exact search of Section 4 came from it (Section 3).
- *A repaired search.* The best exact algorithm for MOSP, the search of Chu and Stuckey (2009), is also a pathwidth algorithm. Two of its published dominance theorems are false, as are their later restatements (Chu, 2011; Fink, 2012). The correct rule was already known in the exact pathwidth literature, and we show that it can be tested by one bipartite matching per candidate. The repaired search is proved sound. Over each benchmark set it changes the total number of search nodes by at most 2%, and no wrong answer of the published rules on a whole instance is known (Section 4).
- *Benchmarks re-certified, and a dataset.* The repaired search re-proved all 115 benchmark optima that rested on the published rules, and none changed (as of 6 October 2026). We release 17,714 graphs, distinct up to isomorphism, from twenty benchmark collections of the twelve problems; 16,087 have a certified pathwidth with a layout that attains it, and so a certified value for every exact problem in the table (Section 5).

How to read the paper. Section 2 states the twelve problems and proves what is true. Section 3 measures which names and communities the literature uses. Section 4 reads the exact open-stacks search as a pathwidth algorithm, finds the two false rules and proves the repaired search sound. Section 5 reports its cost and the dataset, and Section 6 answers the question above. Every theorem we state is machine-checked in the Lean 4 proof assistant. The reader need not read Lean: the proofs are given in the text, and Appendix A maps each result to its formal statement. Appendix B holds the supporting material, Appendix C the bibliometric method, and the data, code and proofs are described at the end of the paper (page 21).

2 The equivalences, proved

This section answers the first half of our question: which rows of the Linhares–Yanasse table are true, and in what sense. Section 1 separated two readings of “plus or minus one”. We call a row *exact* when it meets the fixed-offset reading: its value is pathwidth, or pathwidth plus one, on every input. We call it a *band* when it meets only the literal reading: its value stays within a fixed distance of pathwidth, and both ends of that range occur. We call it *false* when it meets neither. The answer is that eight rows are exact, two are bands and two are false. Section 2.1 states the problems. Section 2.2 proves the answer in four steps: the hub theorem, which ties MOSP to pathwidth; the other exact rows; the bands; and the false rows. Table 2.2 and Figure 2.1 summarise it.

All theorems below are verified in the Lean proof assistant, unless the text says otherwise; Appendix A lists the formal statements. Where a result is published, we cite the published proof and do not repeat it.

Notation. Graphs are finite and simple, $G = (V, E)$, $n = |V|$. A *layout* is a bijection $L : V \rightarrow \{1, \dots, n\}$. For a 0/1 matrix M with rows R (customers, nets, piece types) and columns C (patterns, gates, products), the *MOSP graph* G_M has vertex set R , with $r \sim r'$ when some column has a 1 in both rows; each column becomes a clique on its rows. For an order π of the columns, row r is *active* at position j if it has 1s in columns x, y with $\pi(x) \leq j \leq \pi(y)$.

2.1 The problems

Each problem is a decision problem with an instance and an integer k ; its value is the least k for which the answer is yes. Four of the twelve carry the rest of the paper, and we define them in full. Figure 1.1 shows each on one instance.

Pathwidth $\text{pw}(G)$. Is there a sequence of bags $X_1, \dots, X_r \subseteq V$ covering V , with every edge inside some bag, $X_i \cap X_\ell \subseteq X_j$ whenever $i \leq j \leq \ell$, and $|X_i| \leq k + 1$? (Kinnersley, 1992, p. 346). The condition on $X_i \cap X_\ell$ is the *interval property*: the bags that contain a vertex are consecutive.

Vertex separation $\text{vs}(G)$. Is there a layout L such that for every $i < n$ at most k vertices u with $L(u) \leq i$ have a neighbour v with $L(v) > i$? (Kinnersley, 1992, p. 346).

MOSP $Z(M)$. Given a 0/1 matrix M of customers by patterns, is there an order of the patterns with at most k customers active at every position? (Linhares and Yanasse, 2002, eqs. (1)–(2)).

Gate matrix layout $t(M)$. Given a net–gate matrix, is there an order of the gates and an assignment of nets to k tracks such that nets active at a common position get different tracks? (Möhring, 1990, p. 18); (Wing et al., 1985, Problem 1).

The other eight problems are needed only to read the table. Table 2.1 gives each in one line; Appendix B.1 gives the formal definitions, with the sources.

Table 2.1: The other eight problems of the Linhares–Yanasse table, in one line each. Each value is the least k that is possible. Formal definitions are in Appendix B.1.

Problem	In one line
One-dimensional logic	Order the gates of a circuit so that the intervals spanned by the nets overlap at most k at any point. Its graph is the <i>net graph</i> H , which joins two nets that share a gate. A boundary variant pins two gates to the ends.
PLA folding $\text{pla}(M)$	Gate matrix layout in at most k tracks, with at most two nets on a track. With no limit per track (<i>multiple folding</i>) it is gate matrix layout itself.
Interval thickness $\theta(G)$	Add edges to G to make an interval graph whose largest clique has at most k vertices.
Node search $\text{ns}(G)$	Searchers are placed on and removed from vertices. An edge is cleared when both its ends hold a searcher, and becomes contaminated again if a searcher-free path joins it to a contaminated edge. Clear every edge with at most k searchers at once. $\text{mns}(G)$ is the same with no recontamination allowed.
Edge search $\text{es}(G)$	Node search with a third move: sliding a searcher along an edge clears it. $\text{pes}(G)$ is the same over <i>progressive</i> strategies, which never recontaminate an edge.
Narrowness $\nu(G)$	Add the vertices one at a time in some order, and drop each as soon as all its neighbours have been added. Never hold more than k .
Split bandwidth $\text{sb}(G)$	Repeatedly split a vertex into an edge, sharing its neighbours between the two ends, so that the result has a layout in which adjacent vertices are at most k apart (its <i>bandwidth</i>).
Edge separation	The table’s reference (Lengauer, 1981) supports three readings: cutwidth $\text{cw}(G)$, the most edges crossing a gap of a layout; modified cutwidth $\text{mcw}(G)$, the most edges passing strictly over a vertex; and Lengauer’s vertex separator game $\text{VSG}(G)$, a pebbling game that counts the unpebbled vertices adjacent to pebbled ones.

2.2 What is true

Table 2.2 gives the answer row by row, and Figure 2.1 draws it as a graph around pathwidth. The rest of this subsection proves it.

The hub. Two theorems tie the table to pathwidth. The first joins the two graph rows, vertex separation and pathwidth. The second joins MOSP to pathwidth, and through it all four matrix problems. The second rests on one lemma about path decompositions. For a vertex v , let $\text{first}(v)$ and $\text{last}(v)$ be the least and greatest indices of bags containing v ; by the interval property, $v \in X_i$ exactly when $\text{first}(v) \leq i \leq \text{last}(v)$.

Lemma 2.1 (cliques sit in a bag). *In a path decomposition of G , every clique of G lies inside one bag (Appendix A, A.1).*

Proof. The bags containing a vertex u of the clique form the interval $[\text{first}(u), \text{last}(u)]$. Two vertices of the clique are adjacent, so their edge lies in a bag and their intervals meet. Pairwise intersecting intervals of a line have a common point (the Helly property): let a be the largest left end; every right end is at least every left end, so at least a , and a lies in all of them. So all the intervals share an index i , and the clique lies in X_i . $\square$

The first theorem is known. It says the two graph rows of the table are the same quantity.

Theorem 2.2 (vertex separation is pathwidth; known). *For every graph, $\text{vs}(G) = \text{pw}(G)$ (Appendix A, A.2).*

This is Theorem 3.1 of Kinnersley (1992). Both directions are constructions. From a layout of separation k , each bag is the set counted at a cut together with the next vertex. From a decomposition, ordering the vertices by first gives a layout of the same width.

Table 2.2: The Linhares–Yanasse table, settled. Eight rows are exact, two are bands and two are false, and every true relation is a relation to pathwidth. *Relation*: what is true and proved in Lean, on the graph the problem lives on (the MOSP graph or the net graph H for the matrix problems, the input graph otherwise). *Published source*: the published work the row rests on; the Lean proofs are listed in Appendix A. *Status*: *exact*, a fixed offset; *band*, an inequality whose ends both occur; *false*, the table’s claim fails, with a counterexample family proved in Lean.

Problem	Published source	Relation	Status
MOSP	Yanasse (1997a); Fellows and Langston (1989); Linhares and Yanasse (2002)	$Z(M) = \text{pw}(G_M) + 1$ (M has a 1)	exact
Gate matrix layout	Möhring (1990)	$t(M) = Z(M) = \text{pw}(G_M) + 1$	exact
One-dim. logic	Ohtsuki et al. (1979)	$\text{tracks} = \theta(H) = \text{pw}(H) + 1$; boundary variant $\geq \text{pw}(H) + 1$	exact
PLA folding	Möhring (1990)	$\max(\text{pw} + 1, \lceil N /2 \rceil) \leq \text{pla}$, gap unbounded; multiple folding $= \text{pw} + 1$	false; variant exact
Interval thickness	Möhring (1990); Kirousis and Papadimitriou (1985)	$\theta = \text{pw} + 1$ ($V \neq \emptyset$)	exact
Node search	Kirousis and Papadimitriou (1985, 1986); Bienstock and Seymour (1991)	$\text{ns} = \text{mns} = \text{vs} + 1 = \text{pw} + 1 = \theta$ ($E \neq \emptyset$)	exact
Edge search	Ellis et al. (1994); Kirousis and Papadimitriou (1986)	$\text{vs} \leq \text{es} \leq \text{vs} + 2$, also for pes	band
Narrowness	Kornai and Tuza (1992)	$\text{v} = \text{pw} + 1$ ($V \neq \emptyset$)	exact
Split bandwidth	Fomin (1998)	$\text{pw} \leq \text{sb} \leq \text{pw} + 1$	band
Path-width	Kinnersley (1992)	the reference	exact
Edge separation ^a	Lengauer (1981)	cw, mcw unbounded against pw (stars); $\text{VSG} = \max(1, \text{vs})$	false; VSG exact
Vertex separation	Kinnersley (1992)	$\text{vs} = \text{pw}$	exact

^aAlso misattributed: what Lengauer (1981) proves exactly is the vertex separator game, which is the table’s vertex separation row.

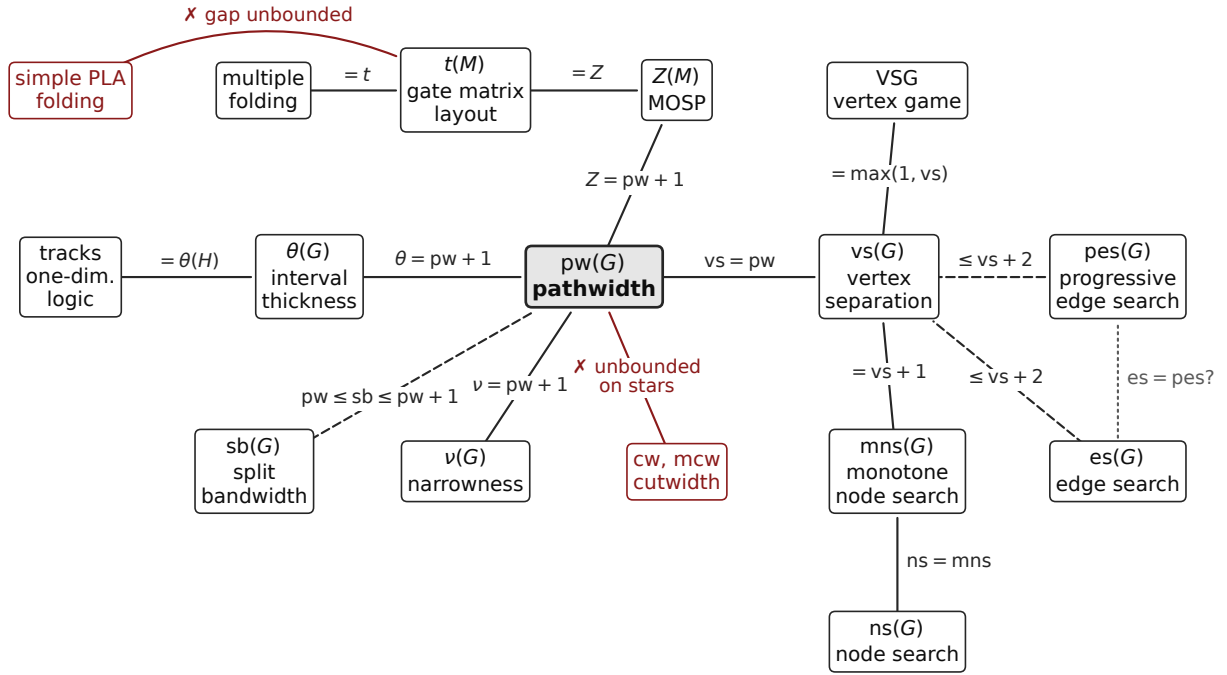


Figure 2.1: Pathwidth is the hub of the table: every exact relation is one or two steps from it, and the only failures are the two false claims, in dark red. Solid edges are exact relations and dashed edges bands, both proved in Lean; the dotted edge is LaPaugh's $\text{es} = \text{pes}$, the one relation not proved, which no row needs. Each false claim is refuted by a counterexample family proved in Lean. The four matrix problems (top left) live on the MOSP graph G_M or the net graph H . Hypotheses are omitted: M has a 1, $V \neq \emptyset$, or G has an edge, as in Table 2.2. Two edges that follow from the others, $\text{mns} = \theta$ and $\text{ns} = \text{vs} + 1$, are not drawn.

The second theorem is the one the rest of the paper uses. In Figure 1.1(d), the sets of customers active at each step of the good order are the bags of a path decomposition of the MOSP graph. The theorem says that this always works, in both directions.

Theorem 2.3 (MOSP is pathwidth plus one). *If M has at least one 1, then $Z(M) = \text{pw}(G_M) + 1$ (Appendix A, A.3).*

Proof. ($\leq$) Take a decomposition of G_M of width k . The rows of each column form a clique, so by Lemma 2.1 they lie in a bag $X_{b(c)}$. Order the columns by $b(c)$. If row r is active at the position of column c , it has columns x, y with $b(x) \leq b(c) \leq b(y)$, and $r \in X_{b(x)} \cap X_{b(y)}$, so the interval property gives $r \in X_{b(c)}$. At most $k + 1$ rows are active there.

($\geq$) Take an order with at most Z active rows at every position. The active sets, with a singleton bag for each row with no 1, form a path decomposition: rows sharing a column are both active at its position, and a row is active on an interval of positions. Its width is $Z - 1$, using $Z \geq 1$. $\square$

For the table, this settles the MOSP row as exact, with offset one, on the MOSP graph. Read as a cutting problem, Theorem 2.3 is Proposition 5 of Yanasse (1997a), which turns each pattern into a clique on its piece types. Read as a layout problem, it is Theorem 7 of Fellows and Langston (1989), with the column-expansion Lemma 4.1 of Fellows and Langston (1987). Linhares and Yanasse (2002, Prop. 2) identify MOSP with gate matrix layout. The proof above needs no assumption that the instance is reduced.

The graph matters. The MOSP literature also uses a second graph, with the patterns as vertices, joined when they share a customer, to split an instance into independent parts (Yanasse and Senne, 2010). It is natural to expect its pathwidth to measure the open stacks as well; it does not, in either direction. One pattern shared by four customers who each have a private pattern has $Z = 4$, while its pattern graph is the star $K_{1,4}$, of pathwidth 1 (Appendix A, A.4). A single customer with many patterns errs the other way.

The exact core. The other exact rows attach to Theorems 2.2 and 2.3 by published arguments. We do not reprove them on paper; each item below cites the published proof, and the Lean proofs are listed in Appendix A. We call the problems that equal pathwidth or pathwidth plus one on every input the *exact core*: the eight exact rows, with multiple folding and the vertex separator game.

Theorem 2.4 (the exact core).

1. *Gate matrix layout is MOSP on the same matrix: if M has a 1, $t(M) = Z(M) = \text{pw}(G_M) + 1$. For a fixed gate order the fewest tracks equals the most nets active at one position, by the left-edge algorithm (Möhring, 1990, p. 31) (Appendix A, A.5).*
2. *Interval thickness: if $V \neq \emptyset$, $\theta(G) = \text{pw}(G) + 1$ (Möhring, 1990, Prop. 3.5) (Appendix A, A.6).*
3. *One-dimensional logic: if every net lies on a gate, the fewest tracks is $\theta(H)$, and if some gate connects a net it is $\text{pw}(H) + 1$ (Ohtsuki et al., 1979, Thm 3) (Appendix A, A.7).*
4. *Narrowness: if $V \neq \emptyset$, $v(G) = \text{pw}(G) + 1$, and for each order σ , $v(\sigma) = \text{vs}(\sigma^{\text{rev}}) + 1$ (Kornai and Tuza, 1992, Prop. 3.1) (Appendix A, A.8).*
5. *Node search: for every graph $\text{ns}(G) = \text{mns}(G)$, and if $E \neq \emptyset$, $\text{ns}(G) = \text{vs}(G) + 1 = \text{pw}(G) + 1 = \theta(G)$ (Kirkousis and Papadimitriou, 1985, Thm); (Kirkousis and Papadimitriou, 1986, Thms 2.3, 4.1) (Appendix A, A.9).*
6. *Multiple folding: the fewest tracks is $t(M) = \text{pw}(G_M) + 1$ (Möhring, 1990, Thm 3.14) (Appendix A, A.10).*

7. *Lengauer’s vertex separator game*: $\text{VSG}(G) = \max(1, \text{vs}(G))$ ([Lengauer, 1981, Thm 4](#)) ([Appendix A, A.11](#)).

For the table, this makes the gate matrix layout, one-dimensional logic, interval thickness, node search and narrowness rows exact, and gives an exact variant of each false row. A single certified pathwidth therefore answers every problem in the exact core; [Section 5.2](#) uses this.

Bands. Two rows are not fixed offsets of pathwidth, but stay within a fixed distance of it.

Theorem 2.5 (bands). *For every graph, $\text{pw}(G) \leq \text{sb}(G) \leq \text{pw}(G) + 1$ ([Fomin, 1998, Thm 8](#)), and $\text{vs}(G) \leq \text{es}(G) \leq \text{vs}(G) + 2$, and the same for pes ([Ellis et al., 1994, Thm 2.1](#)) ([Appendix A, A.12](#)).*

Every end of both bands occurs. K_2 has $\text{sb} = \text{pw} = 1$ and $\text{es} = \text{vs} = 1$; the star $K_{1,3}$ has $\text{pw} = 1$ and $\text{sb} = 2$, and $\text{vs} = 1$ with $\text{es} = 2$; and $K_{3,3}$ has $\text{vs} = 3$ with $\text{es} = 5$. These values are computed by brute force from each quantity’s definition, not proved in Lean.¹ [Fomin \(1998\)](#) assumes a connected graph with at least two vertices; the Lean proof needs neither. LaPaugh’s equality $\text{es} = \text{pes}$ ([LaPaugh, 1993](#)) is the one relation we state and do not prove, and no row needs it.

For the table, this means the following. Relative to the number of open stacks $Z = \text{pw} + 1$, split bandwidth lies in $[Z - 1, Z]$ and edge search in $[Z - 1, Z + 1]$. Both stay within one of Z , so they meet the literal reading of “plus or minus one”. Since both ends occur, neither is a fixed offset, so neither meets the fixed-offset reading.

False rows. Two rows fail even the literal reading: no constant bounds their distance from pathwidth.

Theorem 2.6 (two rows are false).

1. *PLA folding*: for a matrix with a 1, $\max(\text{pw}(G_M) + 1, \lceil |N|/2 \rceil) \leq \text{pla}(M)$, and the gap to $\text{pw} + 1$ is unbounded. The 5×5 identity needs 3 tracks while $\text{pw}(G_M) + 1 = 1$, and the path P_n as a matrix, one gate per edge, has $t = 2$ and $\text{pla} \geq \lceil n/2 \rceil$, so the gap is unbounded on connected instances too ([Möhring, 1990, Prop. 3.15](#)) ([Appendix A, A.13](#)).
2. *Edge separation read as cutwidth*: $\text{pw} \leq \text{cw}$ always, but $\text{cw}(K_{1,n}) \geq n/2$ and $\text{mcw}(K_{1,n}) \geq n/2 - 1$ while $\text{pw}(K_{1,n}) = 1$; so neither cutwidth reading is within a constant of pathwidth ([Appendix A, A.14](#)).

For the table, these rows are false as stated. Each has an exact variant ([Theorem 2.4](#)): the PLA claim holds for multiple folding, and the edge-separation reference proves exactly the vertex separator game.

Several published equalities need a nondegeneracy hypothesis, and one published proof has a gap that we repair; [Appendix B.2](#) gives the details. [Appendix B.3](#) adds a thirteenth problem, outside the table, to the exact core.

Summary. Eight of the twelve rows of the Linhares–Yanasse table are exact, counting the pathwidth row itself, which is the reference. The other seven are pathwidth, or pathwidth plus one, exactly: MOSP, gate matrix layout, one-dimensional logic, interval thickness, node search, narrowness and vertex separation. So are two variants of the false rows, multiple folding and Lengauer’s vertex separator game. Split bandwidth and edge search are bands of width one and two, within one of the number of open stacks but not at a fixed offset. Simple PLA folding and edge separation are false as stated. One theorem is stated and not proved, LaPaugh’s $\text{es} = \text{pes}$, and no row needs it. One of the table’s references, Kashiwabara and Fujisawa (1979), is not held; its row is proved from [Möhring \(1990\)](#) and [Kirosis and Papadimitriou \(1985\)](#).

¹ $\text{sb}(K_{1,3}) \geq 2$ is also argued by hand: splits of a tree are trees and keep at least three leaves.

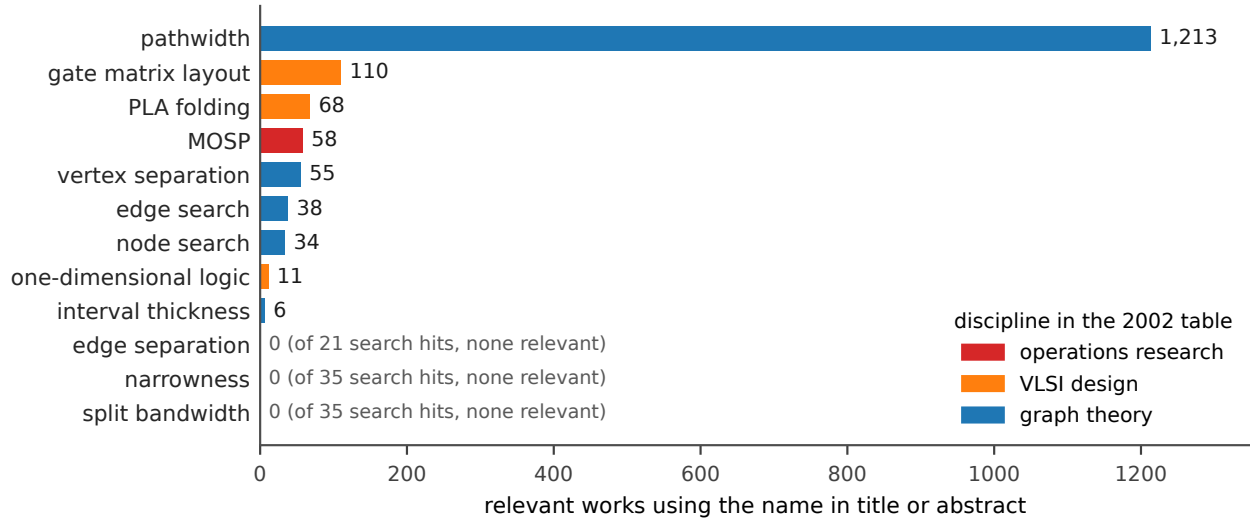


Figure 3.1: Pathwidth has more relevant works than the other eleven names together, three times over, and three names have none. Works using each problem’s name, in title or abstract, in the sense of the Linhares–Yanasse table; all years to September 2026. Colour gives the discipline that the table assigns the problem.

3 The names and the communities

Section 2 showed that eight of the twelve rows of the Linhares–Yanasse table are one quantity. A result proved under any of their names therefore holds under all of them. But a reader who searches under one name will not find a result published under another. This section asks two questions about the literature. Which name does it use for the quantity? And, besides that name, which community has been most active? The answers are pathwidth, by far, and open stacks. The second answer explains where the best exact algorithm for the family came from, and leads to Section 4.

How we counted. We searched OpenAlex (Priem et al., 2022) on 29 September 2026 for each problem’s name and its common variants, in titles and abstracts, within Computer Science, Mathematics, Engineering and Decision Sciences. A search hit need not use the name in the table’s sense: “narrowness” matches term rewriting, and “path-width” matches roads and tool paths. So we read the 2,271 hits and kept the 1,593 that are about one of the twelve problems. The 700 hits for the eleven smaller names were judged one by one, by one annotator with model assistance; the 1,571 pathwidth hits were filtered by a stated rule, checked on samples. A mechanical phrase rule agrees with the judgements on 78% of the 700 and leads to the same conclusions. The counts are lower bounds, since a work can study a problem without naming it in its abstract. Appendix C gives the method in full, the robustness check and the counts per name.

3.1 Pathwidth has absorbed the family

Figure 3.1 gives the counts. Pathwidth has 1,213 relevant works, 21 times as many as MOSP and more than three times as many as the other eleven names together (380). Three names have no relevant work at all: narrowness, split bandwidth and edge separation. Two more are rare: interval thickness has 6 works and one-dimensional logic 11. Pathwidth is also the only name still growing: its 311 works in 2020–2024 are more than the whole history of any other name (Appendix C). So anything one would want to know about algorithms for this class of problems was most likely published under “pathwidth”.

Table 3.1: Besides pathwidth, the most active name over 2005–2024 is MOSP, although vertex separation led in 2015–2019. Relevant works per five-year period, for the three names with the most works over 2005–2024; the next are edge search (28) and node search (20).

Name	2005–09	2010–14	2015–19	2020–24	2005–24
Pathwidth	114	179	267	311	871
MOSP	17	17	5	12	51
Vertex separation	7	10	15	0	32

Table 3.2: The open-stacks papers are read apart from graph theory. Works citing the papers that the Linhares–Yanasse table cites, by the discipline of the papers they cite. Of 135 works citing an operations-research paper, 6 also cite a graph-theory paper (4%); of 285 citing a VLSI paper, 65 do (23%). The 2002 paper itself counts as operations research. OpenAlex, 29 September 2026; 844 citing works in all.

Works citing	All	... of which also cite		
		operations research	VLSI	graph theory
operations research	135	–	14	6
VLSI	285	14	–	65
graph theory	507	6	65	–

3.2 Besides pathwidth, open stacks

The VLSI names belong to the 1980s: since 2005 gate matrix layout has 9 relevant works and PLA folding 6. Table 3.1 shows the three names with the most works over 2005–2024. MOSP is second, with 51, ahead of vertex separation (32) and edge search (28). Vertex separation led only in 2015–2019. The open-stacks community, in operations research, is thus the one most active besides pathwidth.

It has also worked largely apart from graph theory. Table 3.2 counts the 844 works that cite the papers of the table, by the disciplines of the papers they cite. Only six works cite both an open-stacks paper and a graph-theory paper. Two are by the table’s authors, one is open-stacks work on graph properties, and three are pathwidth or vertex-separation work from computer science. The VLSI papers are read with graph theory about five times as often.

3.3 From an open-stacks challenge to a pathwidth solver

It is not a coincidence that the best exact algorithm for MOSP came from the open-stacks community and is also, on the published evidence, among the strongest exact algorithms for pathwidth. The account is in [Chu and Stuckey \(2009, §1\)](#). MOSP was the subject of the first Constraint Modelling Challenge ([Smith and Gent, 2005](#)), posed in May 2005, which drew 13 entries. The winning entry, by García de la Banda and Stuckey, combined dynamic programming with branch and bound and was an order of magnitude faster than any other ([García de la Banda and Stuckey, 2007](#)). [Chu and Stuckey \(2009\)](#) built on it. They branched on which customer’s stack closes next rather than on which product is made next, added nogood recording and dominance rules, and became five to six orders of magnitude faster on hard instances. Their solver closed every open instance of the challenge within ten seconds. MOSP’s busiest decade in Table 3.1, 2005–2014, is the decade of this work.

The customer search reads an instance only through its MOSP graph ([Chu and Stuckey, 2009, p. 4](#)), so it decides pathwidth (Section 4). Ported to graphs, it proves more instances than reported on the benchmarks where published exact pathwidth results exist, those of [Coudert et al. \(2016\)](#). It proves the pathwidth of 11,194 of the 11,534 Rome graphs (97.1%), against their 95.6%, both within 600 seconds per graph. It

proves all 50 VSPLIB trees, against their 30, and 39 of the 73 Harwell–Boeing graphs, against their 26. These are comparisons with published numbers obtained on other hardware, not a head-to-head run.

Two consequences shape the rest of the paper. First, a result on any of the twelve problems should be stated, and searched for, as a result on pathwidth. Section 4 does this for the customer search. It finds that the search’s first dominance rule is, in correct form, a theorem already known in the pathwidth literature, and that two of its published theorems are false. Second, a benchmark for any problem in the exact core is a pathwidth benchmark, which is how Section 5.2 builds its dataset.

4 An exact open-stacks search, read as a pathwidth solver

The best exact method for MOSP on the standard benchmarks is the *customer search* of Chu and Stuckey (2009). It searches over the orders in which the customers’ stacks close, not over product sequences, and prunes with dominance rules, three of them stated as theorems (their Theorems 1, 2 and 3) and a fourth, the subset rule, in their §2. Chu and Stuckey (2009) place MOSP among “graph path-width and gate matrix layout”, citing Linhares and Yanasse (2002), but do not use pathwidth in the search. By Theorem 2.3 it is nevertheless a pathwidth algorithm: any graph is the MOSP graph of the instance with one customer per vertex and one product per edge, whose optimum is pathwidth plus one.

This section reads the search that way, and finds three things. Two of the published theorems are false, and so are their restatements in Chu’s thesis (Chu, 2011) and, with a stronger premise, in Fink (2012). The repaired first rule is an instance of a known pathwidth theorem, Tamaki’s commitment lemma; the published rule is that lemma with its interior condition dropped. And the whole repaired search is proved sound in Lean.

4.1 The search decides pathwidth

Chu and Stuckey (2009, p. 4) observe that the products matter only through the MOSP graph. So fix a graph $G = (C, E)$ on the customers and an integer $k \geq 0$. For a customer c and a set $T \subseteq C$ of closed customers, write $N[c]$ for the closed neighbourhood of c , and

$$\begin{array}{ll}
 O(T) = \bigcup_{c \in T} N[c] & \text{the stacks opened once } T \text{ has closed,} \\
 \text{fin}(X) = \{c : N[c] \subseteq X\} & \text{the customers free to close when } X \text{ is open,} \\
 \text{cl}(T) = \text{fin}(O(T)) & \text{the free closure of } T, \\
 b(T) = |O(T) \setminus T| & \text{the stacks open after } T \text{ has closed,} \\
 o(c, T) = N[c] \setminus O(T), \quad \text{open}(c, T) = |o(c, T)| & \text{the stacks closing } c \text{ would newly open.}
 \end{array}$$

A move at T closes one customer $c \notin T$, at cost $\text{cost}(T, c) = |O(T \cup \{c\}) \setminus T| = b(T \cup \{c\}) + 1$, the stacks open at that moment counting c ; it is *playable* when $\text{cost}(T, c) \leq k$. Write $P_k(T)$ when some order of $C \setminus T$ has every step playable. On entering a node the search closes every free customer, so it stands only at *states*, sets with $S = \text{cl}(S)$, and the *child* of S by c is $S \cdot c = \text{cl}(S \cup \{c\})$. The search’s predicate $\text{Sol}_k(S)$ holds when $S = C$, or when some playable c has $\text{Sol}_k(S \cdot c)$ (Appendix A, A.15). Every node the search visits satisfies the invariant $b(S) \leq k$.

Lemma 4.1 (free moves never hurt). *If $b(T) \leq k$, then $P_k(T) \iff \text{Sol}_k(\text{cl}(T))$ (Appendix A, A.16).*

Closing a customer whose neighbourhood is open opens nothing, never makes a later step dearer, and leaves one fewer stack open. The hypothesis is needed: closing the free customers one by one costs $b(T)$ at each step.

Theorem 4.2 (the search decides pathwidth). *For nonempty C , $\text{Sol}_k(\emptyset) \iff \text{pw}(G) + 1 \leq k$; on the MOSP graph of an instance M with a requirement, $\text{Sol}_k(\emptyset) \iff Z(M) \leq k$ (Appendix A, A.17).*

Proof. For a closing order $c_1, \dots, c_n$ with prefixes S_i , the i -th step costs $1 + |\partial S_i|$, where ∂X is the set of vertices outside X with a neighbour in X . Read backwards as a layout, the cut separating $C \setminus S_i$ from S_i counts exactly ∂S_i . So the cost of the order is the vertex separation of the reversed layout plus one, and Lemma 4.1 at the root and Theorem 2.2 finish the proof. The Lean statement goes through narrowness (Kornai and Tuza, 1992), which counts the forward order directly. $\square$

So a refutation, $\neg \text{Sol}_k(\emptyset)$, may only mean $\text{pw}(G) > k - 1$. The rest of the section asks whether the pruning rules preserve that meaning.

What a rule must satisfy. At a state S the search has a set Q of *old moves* (Section 4.3). Its candidates are the customers outside $S \cup Q$; the *cost cut* keeps the playable ones, P , in index order, and a *filter* keeps a sublist $L \subseteq P$. The filter is *node-sound* at (S, Q) if, whenever every $q \in Q$ has $\neg \text{Sol}_k(S \cdot q)$, $\text{Sol}_k(S)$ implies $\text{Sol}_k(S \cdot c)$ for some $c \in L$ (Appendix A, A.18). Each rule justifies a discarded move r by a *covering*, a move c with $\text{Sol}_k(S \cdot r) \Rightarrow \text{Sol}_k(S \cdot c)$. A node is sound when every link a rule cites is a true covering and every chain of links from a discarded candidate ends in L or in Q ; the second condition can fail through a cycle even when every link is true. The filter runs the definite move, then the subset rule, then the better move, each once.

4.2 The definite move is false as published

Chu and Stuckey (2009, §3.1, p. 6) state:

Theorem 1. Suppose $S \mathrel{++} [q]$ is playable and $\text{close}(q, S) \geq \text{open}(q, S)$, then if $U' = S \mathrel{++} R$ is a solution, there exists a solution $U = S \mathrel{++} [q] \mathrel{++} R'$,

where $\text{close}(q, S)$ counts the customers d with $o(d, S) \subseteq o(q, S)$ (p. 4). The rule keeps q alone. We read d as ranging over customers not yet closed, as the code does; read literally, closed customers count too, which weakens the premise, and both readings are refuted below. Chu (2011, Thm 6.3.6, p. 142) restates the theorem with the same premise and proof, reworded for k stacks. Write $X = S \cdot q$.

Lemma 4.3 (what the premise says). $\text{close}(q, S) = |X \setminus S|$. Hence the premise holds iff $b(X) \leq b(S)$: the child has no more open stacks than the parent (Appendix A, A.19).

Counterexample 4.4. Let G be the graph on customers $0, \dots, 13$ with the 26 edges

$$\begin{aligned} &0-3, 0-4, 0-7, 0-11, 1-2, 1-5, 1-6, 2-3, 2-4, 5-9, 5-11, 5-12, 6-8, \\ &6-13, 7-9, 7-10, 7-13, 8-9, 8-10, 8-11, 9-10, 9-11, 10-11, 10-12, 10-13, 12-13, \end{aligned}$$

with $S = \{2\}$, $k = 6$ and $q = 0$ (Figure 4.1). Then $O(S) = \{1, 2, 3, 4\}$, $o(0, S) = \{0, 7, 11\}$ and $\text{cost}(S, 0) = 6$, so 0 is playable; the customers with $o(d, S) \subseteq \{0, 7, 11\}$ are 0, 3 and 4, so $\text{close}(0, S) = 3 \geq \text{open}(0, S)$; and 0 comes first in index order, so the rule keeps only 0. A solution exists from S : the order

$$1, 3, 4, 6, 12, 13, 0, 5, 7, 8, 9, 10, 11$$

costs at most 6. None exists from the child $X = \{0, 2, 3, 4\}$: only seven sets are reachable from X by moves of cost at most 6, and none is C (Appendix A, A.20).

A counterexample to Chu & Stuckey's Theorem 1, at $k = 6$
 0 is playable and $\text{close}(0, S) \geq \text{open}(0, S)$, so the rule keeps 0 alone. A solution from S exists
 (first moves 1, 3, 4, 6), but none starts with 0.

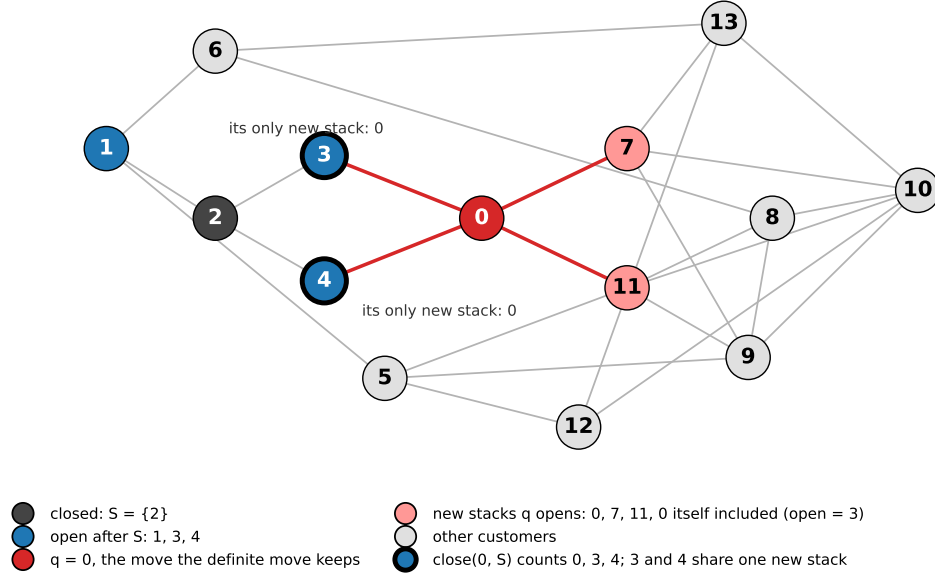


Figure 4.1: The graph of Counterexample 4.4 at the state where the definite move fails. Customer 2 is closed; 1, 3 and 4 are open. Moving 0 opens the new stacks 0, 7 and 11 and closes 0, 3 and 4, so $\text{open} = \text{close} = 3$ and the rule keeps 0 alone. But 3 and 4 have the single new stack 0 between them: closing them first gains two stacks for the price of one, which a solution starting with 0 cannot match.

The published implication, as a universal statement, is refuted in Lean under both readings of close , and so is the thesis's Theorem 6.3.6 (Appendix A, A.21). Appendix B.5 gives the search for smaller counterexamples, and Appendix B.4 the analysis of why the published proof fails. Fink (2012, pp. 27–28) restates the rule with a stronger premise, counting only dominated customers of smaller index than q . The graph above refutes it under no labelling, but a third twin of 3 and 4 gives a 15-customer graph that does (Appendix A, A.22).

The repair. Call q *hereditarily definite* at S when $b(X) \leq b(B)$ for every B with $S \subseteq B \subseteq X$ and $q \notin B$. By Lemma 4.3, the published premise is the case $B = S$. In the counterexample the repair fails at $B = \{2, 3, 4\}$, where $b(B) = 2 < 3 = b(X)$ (Appendix A, A.23).

Lemma 4.5 (open stacks are submodular). *For all $A, B \subseteq C$, $b(A \cup B) + b(A \cap B) \leq b(A) + b(B)$ (Appendix A, A.24).*

Proof. $b(T) = |O(T)| - |T|$, $O(A \cup B) = O(A) \cup O(B)$ and $O(A \cap B) \subseteq O(A) \cap O(B)$; the customers themselves balance exactly. $\square$

Theorem 4.6 (the repaired definite move is sound). *If $q \notin S$ is hereditarily definite at S , then $P_k(S) \Rightarrow P_k(X)$. At a state with the invariant, $\text{Sol}_k(S) \Rightarrow \text{Sol}_k(S \cdot q)$ (Appendix A, A.25).*

Proof. Take an order from S of cost at most k with prefixes T_i , in which q is the j -th step, and follow it from X , skipping customers already in X . A step $i < j$ closes c_i at $T_{i-1} \cup X$. With $Z = T_i$ we have $S \subseteq Z \cap X \subseteq X$ and $q \notin Z$, so the hypothesis gives $b(X) \leq b(Z \cap X)$, and Lemma 4.5 gives $b(Z \cup X) \leq b(Z)$: the step costs no more than the original. After step $j - 1$ the closed set contains T_j with the same opened set, and the rest of the order applies. The search's form follows by Lemma 4.1. $\square$

This theorem is not new. The hereditary premise is the commitment condition of Tamaki, and the proof is the proof of the commitment lemma, [Kitsunai et al. \(2016, Lemma 1, p. 142\)](#); Section 4.5 makes the correspondence exact. What we did not find in the papers we hold is the following cheap form of the test.

Theorem 4.7 (the repair is a matching condition). *Let $Y = X \setminus (S \cup \{q\})$, and join each $d \in Y$ to the stacks of $o(d, S) \subseteq o(q, S)$. Then q is hereditarily definite at S iff this bipartite graph has a matching with at least $\text{open}(q, S) - 1$ edges ([Appendix A, A.26](#)).*

Proof. The sets of the repair are $B = S \cup D$, $D \subseteq Y$, and $b(X) \leq b(S \cup D)$ iff $|D| - |o(D)| \leq \delta$ with $\delta = |Y| + 1 - \text{open}(q, S)$, where $o(D)$ is the union of the $o(d, S)$. By the deficiency form of Hall's theorem the largest matching has size $|Y| - \max_D(|D| - |o(D)|)$, which is at least $|Y| - \delta = \text{open}(q, S) - 1$ exactly when the repair holds. The Lean proof reduces the deficiency form to Mathlib's Hall theorem with δ dummy stacks. $\square$

In words: the rule is safe exactly when all but one of the new stacks can be paired with a different freed customer who needed it. In the counterexample, closing 0 opens three new stacks and frees 3 and 4, both of whom needed only stack 0: one pairing where two are required. Every q with $\text{open}(q, S) \leq 1$ passes at once, so the published rule is right there ([Appendix A, A.27](#)).

4.3 The other rules

The subset rule. “If $o(c_i, S) \subseteq o(c_j, S)$ and $i < j \dots$ move j can be removed” ([Chu and Stuckey, 2009, §2, p. 5](#)). As coded, d dominates r when $o(d, S) \subseteq o(r, S)$ and the inclusion is strict or $d < r$; every dominated playable r is dropped, with a fallback that keeps P if all would go. The covering is sound for every set S : if $o(d, S) \subseteq o(r, S)$ and r is playable, then d is playable and $\text{Sol}_k(S \cdot r) \Rightarrow \text{Sol}_k(S \cdot d)$, because d is free after r and closing d then r reaches the same state at no greater cost. With the index tie-break, domination is a strict order, so an undominated move with a solution survives, and the fallback never fires when a solution exists ([Appendix A, A.28](#)). The tie-break is necessary: without it, two customers with equal new stacks dominate each other and both go, which loses a solution on a 7-customer graph ([Appendix A, A.29](#)).

The better move. [Chu and Stuckey \(2009, §3.2, p. 6\)](#): “Suppose $S \mathrel{++} [q]$ and $S \mathrel{++} [r, q]$ are playable and $\text{close}(q, S \cup \{r\}) \geq \text{open}(q, S \cup \{r\})$ then if $U' = S \mathrel{++} [r] \mathrel{++} R$ is a solution there exists a solution $U = S \mathrel{++} [q] \mathrel{++} R'$.” Its premise is Theorem 1's premise for q at the child $\text{cl}(S \cup \{r\})$, together with playability of r then q , and its conclusion swaps q and r ([Appendix A, A.30](#)). The swap is sound; the appeal to Theorem 1 is not. On the graph of Counterexample 4.4 at the root with $k = 6$, the definite move does not fire, customers 0 and 2 both survive the subset rule, and the premise holds for $r = 2$, $q = 0$, so the rule drops 2 citing 0. But $\{2\}$ has a solution and $\{0\}$ has none ([Appendix A, A.31](#)). At that node another kept customer still has a solution, so this is a false link and not a lost node; no node where the better move loses the last solution has been found. [Chu \(2011, Thm 6.3.8, p. 143\)](#) states the better move with a different premise, $\text{close}(q, S) \geq \text{open}(q, S \cup \{r\})$, and proves it through Theorem 6.3.6. The same graph refutes it at $S = \{2\}$, $r = 3$, $q = 0$ ([Appendix A, A.32](#)). The repair cites q for r only if q is hereditarily definite at $\text{cl}(S \cup \{r\})$, one matching at the child; then the swap argument goes through ([Appendix A, A.33](#)).

Two earlier faults of this rule in our own implementation are now proved in Lean too: a close count that included the customers r finishes on its own, and an order of the rules in which the subset rule could cite a customer the better move had already dropped, which loses a node through a cycle of true links ([Appendix A, A.34](#)). The second is an instance of the general observation of [Beck et al. \(2025, Prop. 3, Example 6\)](#) that mutually dominating transitions must be tie-broken.

The old move. Chu and Stuckey (2009, Thm 3, p. 7) let a child inherit a sibling q whose subtree failed, provided that inserting q before each later step keeps that step playable. The rule is sound as coded: if $\text{cost}(S \cup \{q\}, c) \leq k$, then $\text{Sol}_k(\text{cl}(\{q\} \cup S \cdot c)) \Rightarrow \text{Sol}_k(S \cdot q)$, so a refuted sibling stays refuted along any path whose steps pass the test (Appendix A, A.35). The test is necessary; the smallest graph that shows it is the path on five vertices (Appendix A, A.36).

The memo. A refuted state is recorded and refused when met again. Whether a state with $b(S) \leq k$ has a solution depends on the state alone, not on the path that reached it (Appendix A, A.37). One implementation of ours declined to run the memo beside the old move, on the ground that a refutation obtained with inherited old moves depends on the ancestors. A node taken on its own, with an old move that is not in fact refuted, can indeed record a solvable state (Appendix A, A.38); Theorem B.2 (Appendix B.6) shows that this never happens in a real run.

4.4 The repaired search is sound

The main theorem rests on two lemmas about the filter and about whole runs, Proposition B.1 and Theorem B.2, which Appendix B.6 states and proves.

Theorem 4.8 (the revised search is sound). *Run the customer search with free moves, the memo, old moves and the filter definite $\rightarrow$ subset $\rightarrow$ better, the definite move firing only on a hereditarily definite customer and the better move citing only under the repaired premise. If it refutes k from the root, then no closing order costs at most k , $\text{pw}(G) > k - 1$, and on the MOSP graph of an instance M with a requirement, $Z(M) > k$ (Appendix A, A.39).*

What the Lean model covers is the search as specified; that our C and Python implement it is checked, not proved. A transcription of the Lean filter agreed with the production filter on 16,244,090 node checks over graphs on 1 to 17 vertices; the matching agreed with the hereditary premise on 36,939,226 candidates; and a search written from the theorems agreed with the C, the Python and an exact oracle on 17,431,232 runs over 52,592 graphs on 1 to 17 vertices. Under the published rules the filter is not node-sound (Appendix A, A.40), and in a check of 570,206 runs with memo and old move it lost the last solution at a node in 58 runs, all on graphs built around Counterexample 4.4 and all at k equal to the optimum. Every one found a solution through another branch and answered correctly. No whole instance on which the published rules answer *unsat* wrongly is known.

4.5 The search in pathwidth language

For a set T of vertices write $N(T)$ for its *border*, the vertices outside T with a neighbour inside, and $d(T) = |N(T)|$, in the notation of Kobayashi et al. (2014, p. 390). A vertex sequence is w -feasible when every prefix has $d \leq w$, and $\text{vs}(G)$ is the least w for which some permutation is w -feasible. Table 4.1 translates.

Kobayashi et al. (2014, Algorithm 1, p. 393) decide whether a vertex set extends to a w -feasible permutation: replace the set by a committed extension, answer from a failure table or at V , recurse on each feasible one-vertex extension, record failures. The customer search is this loop with $w = k - 1$: the free closure and the definite move play the part of the commitment, the memo is the failure table, and the cost cut is the feasibility test. Coudert et al. (2014, §3.2) has the same three parts in a branch and bound. Neither line cites the other in the papers we hold: Chu and Stuckey (2009) predates the commitments and both 2014 papers, and none of the pathwidth papers cites it.

Table 4.1: The customer search in the vocabulary of exact pathwidth algorithms. Appendix A gives the formal names.

Customer search	Layout
closing order from $\emptyset$	vertex sequence, read forwards
open stacks $b(T)$	border size $d(T)$
$\text{cost}(T, c)$	$d(T \cup \{c\}) + 1$
$\text{Sol}_k(\emptyset)$	$\text{vs}(G) \leq k - 1$
free closure $\text{cl}(T)$	full set of T (Suchan and Villanger, 2009, p. 328)
memo	failure table
definite move, repaired	commitment to the child

Commitments. Kitsunai et al. (2016, p. 142), stating Tamaki’s notion, call an extension τ of σ *commit-table* when $d(X') \geq d(V(\tau))$ for every X' with $V(\sigma) \subseteq X' \subseteq V(\tau)$; their Lemma 1 says that if σ extends to a feasible permutation, so does τ .

Lemma 4.9. *For $q \notin S$ and $X = \text{cl}(S \cup \{q\})$: the published premise holds iff $d(X) \leq d(S)$, the commitment condition at the single set $X' = S$; and the repaired premise holds iff X is a commitment from S , the condition at every X' between them (Appendix A, A.41).*

So Theorem 4.6 is the commitment lemma at the child, and the published Theorem 1 is a commitment whose interior condition was dropped. In Counterexample 4.4 the endpoint test passes with $d(X) = 3 = d(S)$, while the interior set $\{2, 3, 4\}$ has border 2 (Appendix A, A.42). The published premise is not wasted: by Kitsunai et al. (2016, Lemma 10, Cor. 2, pp. 148–149) it guarantees a commitment to the least-border set between S and X , which need not contain q (Appendix A, A.43). In the counterexample that set is $\{2, 3, 4\}$ (Appendix A, A.44). Committing there when the matching fails is a second repair that gives up no pruning; we have not built it.

A commitment of depth one, S to $S \cup \{q\}$, holds iff $\text{open}(q, S) \leq 1$ (Appendix A, A.45). The greedy step of Coudert et al. (2014, Lemma 3, p. 49) is exactly that condition (Appendix A, A.46), and the push rule of Suchan and Villanger (2009, Rule 1, p. 330) is a case of it. So every published pathwidth rule of this kind lies in the region $\text{open} \leq 1$, where the published definite move is right; Kobayashi et al. (2014, p. 388, Table 2) report depth-one commitments as “extremely effective”. Chu and Stuckey’s Theorem 1 reaches into $\text{open} \geq 2$, and that is where it fails. For the cost of the test, Kitsunai et al. (2016, Cor. 2) find the least border between the ends of a given extension by a minimum s – t separator; Theorem 4.7 is that computation for the single target X , where it becomes a bipartite matching.

Table 4.2 sums up what is known, limited to the pathwidth papers we hold (Suchan and Villanger, 2009; Bodlaender et al., 2012; Coudert et al., 2014, 2016; Kobayashi et al., 2014; Kitsunai et al., 2016). “None found” is not a claim of priority.

Prior reports. We looked for any earlier report of the error. We listed the works citing Chu and Stuckey (2009), Chu’s thesis and its IJCAI 2013 abstract in OpenAlex, Semantic Scholar and OpenCitations: 141 works. Of the 38 that cite the CP paper we read 24 in full (63%), and of the 103 that cite only the thesis, 32 (31%). No work read gives a counterexample, an erratum or a correction. One, Fink (2012), restates the definite move as true with a stronger premise, refuted above. Three use an implementation of the rules and report no failure; among them Frinhani et al. (2018) ran the original code to obtain the optima that Martin et al. (2022) reuse. We found no prior report of the error. The pathwidth literature has the correct form of the rule, as a commitment, and none of the pathwidth papers we hold mentions MOSP.

Table 4.2: Each rule of the customer search against the exact-pathwidth literature.

Rule	Published counterpart	Status
free move	full sets (Suchan and Villanger, 2009); Kitsunai et al. (2016, Prop. 3)	known
definite move, open ≤ 1	depth-one commitments; greedy step; push rule	known
definite move, published	–	false (Cex. 4.4)
definite move, repaired	commitment lemma (Kitsunai et al., 2016, Lemma 1)	known; matching test (Thm 4.7) not found
subset rule	none found	proved here
better move, published	none found	false
better move, repaired	none found	proved here
old move	none found	proved here
memo	subset DP (Bodlaender et al., 2012); prefix and failure tables	known; with the old move, proved here (Thm B.2)

5 Computational results and a dataset

5.1 The revised algorithm in practice

The revised search changes two tests. The definite move fires on the first playable q that passes the published test and then the matching of Theorem 4.7; a q that fails the matching is passed over. The better move cites q for r only if the same matching succeeds at $\text{cl}(S \cup \{r\})$. The matching joins at most close -1 customers to open stacks and stops at open -1 edges. Since 1 October 2026 these are the default in both of our solvers, the MOSP customer search and the graph pathwidth solver, in C and in Python; the published rules remain behind a flag.

Table 5.1 gives the cost in nodes. The repair changes tree sizes by at most a few percent and never changed an answer. Per node it costs about 2% in the MOSP C engine, measured on six hard instances at 75–125 customers. The published test passes and the matching then fails on 0.07–0.27% of definite-move candidates, a rate that rises with size.

Re-certification. Of the 6,374 certified optima in our MOSP corpus, 6,259 do not depend on the customer search: they rest on an exact subset-lattice computation, a checked DRAT refutation, an earlier SAT search, or a lower bound equal to the value. The other 115 rested on the customer search under the published rules. The repaired solver has re-refuted the value minus one on all 115 of them (as of 6 October 2026), and no value changed. Table 5.2 gives the seven hardest, 125-customer instances at densities 2 and 4, which needed a split of the root over many cores. The table was read on 3 October 2026, when three were still running; all three have since been refuted. The split is proved sound (Appendix A, A.53).

Certificates. A refutation by the search is a claim about a tree of 10^{11} nodes at 125 customers, and nothing in Theorem 4.8 lets a reader check that the code ran the search it models. So the search emits a certificate: the tree, every pruning step with its rule and witness, and for each definite and better step the matching of Theorem 4.7. An independent checker, which shares no code with the search, recomputes every premise from the matrix, verifies each matching (distinct customers, distinct stacks, each stack newly opened by its customer, at least open -1 edges), requires every chain of covers to end at a child or an old move with no cycle, and replays the tree. Each accepted link is a Lean theorem; that an acyclic covering by such links is sound is a short paper argument, not yet formal. Table 5.3 gives the results on the corpus at 9–75 customers.

Table 5.1: The cost of the repair: nodes under the published and the repaired rules, in paired runs of the same decision in one worker. Totals over the pairs that finished under both settings; a censored pair says nothing about cost. MOSP rows refute the optimum minus one; the graph row is whole descents at 120 s each.

Where	Vertices	Pairs finished	Nodes, published	Nodes, repaired	Ratio	Worst pair
MOSP corpus, default	9–40	6,135 of 6,135	228,147	228,154	1.00003	1.009
MOSP corpus, csearch	9–40	6,135 of 6,135	202,962	202,971	1.00004	1.040
Chu & Stuckey classes	50–100	118 of 125	1,358,096,671	1,362,205,210	1.00303	1.013
Chu & Stuckey classes, cap 2×10^8 nodes	125	11 of 23	90,969,508	90,961,419	0.99991	1.000
Graph pathwidth, whole descents	4–2,916	731 of 880	2,977,849,807	3,038,670,717	1.020	–

Table 5.2: Re-certification of the hardest corpus values under the repaired rules, by a split of the root into independent tasks. k : the value minus one, which must be refuted. For an instance in flight, the nodes and core-hours are those of the finished tasks so far.

Instance	Value	k	State	Tasks refuted	Nodes	Core-hours
Random-125-125-4-1_0	57	56	refuted	669	4.97×10^{10}	18.1
Random-125-125-2-4_0	24	23	refuted	1,686	4.98×10^{11}	154.1
Random-125-125-4-5_0	46	45	refuted	616	7.96×10^{10}	26.1
Random-125-125-4-2_0	57	56	refuted	842	7.64×10^{10}	30.2
Random-125-125-2-1_0	24	23	in flight	197	8.24×10^{10}	51.0
Random-125-125-2-5_0	20	19	in flight	168	6.97×10^{10}	45.6
Random-125-125-4-4_0	51	50	in flight	745	1.28×10^{11}	67.1

Every refutation emitted within 60 s verifies, 6,275 of 6,286 under the `csearch` configuration, and none is rejected; the 35.5 MB of certificates check in 147 s. On the graph of Counterexample 4.4 rooted at $\{2\}$, the published rules emit an exhaustive tree that claims no solution; a checker of the published premises accepts it, and ours rejects it, even with a largest matching attached (one edge where two are needed).

5.2 A dataset with certified pathwidth

Section 2 makes one certified pathwidth an exact answer to every problem in Table 1’s exact core: pathwidth, vertex separation and the vertex separator game take the width w ; MOSP, gate matrix layout with multiple folding, one-dimensional logic, interval thickness, narrowness and node search take $w + 1$. We therefore built one dataset from every benchmark collection of those problems that we could obtain, and the MOSP corpus. The unit is the isomorphism class of the graph, the MOSP graph for a matrix instance: two instances with isomorphic graphs have the same value under every problem, so the dataset holds one record per class, with its members and their isomorphism maps. Each record carries the graph, the width, its provenance (*refutation*, *bound* or *upper bound*), a witness layout of exactly that width and, for a bound, a minor certificate that replays.

Table 5.4 gives the counts. Twenty collections, 21,754 instance files, reduce to 17,714 classes: the MOSP collections (Smith and Gent, 2005; Faggioli and Bentivoglio, 1998; Chu and Stuckey, 2009; de Carvalho and Soma, 2015; Frinhani et al., 2018), eleven VLSI gate matrix circuits (de Oliveira and Lorena, 2002), the VSPLIB sets (Duarte et al., 2012) and the Small graphs (Martí et al., 2008), the Rome graphs (Di Battista et al., 1997), and treewidth collections used as pathwidth benchmarks (van den Broek and Bodlaender, n.d.; Larisch and Salfelder, n.d.; Dell et al., 2017, 2018). Of the classes, 16,087 carry a certified pathwidth with a witness layout and 862 a witnessed upper bound. An independent checker, sharing no code with the solvers, re-reads all 20,987 member files with its own readers, maps each onto its record by isomorphism, re-measures every witness and replays every bound certificate, and reports no failure on the

Table 5.3: Certificates for the repaired rules on the certified corpus at 9–75 customers, refuting the optimum minus one, 60 s emission budget. *Unknown*: emission did not finish. Below 41 customers the `default` rows match `csearch` and are omitted.

Customers	Config.	Inst.	Verified	Rejected	Unknown	Definite	Better	Matching edges	gz bytes (median)	Check ms (median)
9–10	<code>csearch</code>	1,614	1,614	0	0	307	92	32	332	0.04
11–20	<code>csearch</code>	2,508	2,508	0	0	2,657	1,171	658	356	0.08
21–30	<code>csearch</code>	1,816	1,816	0	0	14,796	6,148	5,675	414	0.24
31–40	<code>csearch</code>	197	197	0	0	46,337	18,176	17,150	650	1.27
41–50	<code>csearch</code>	120	120	0	0	530,424	120,302	152,597	12,045	33.24
51–60	<code>csearch</code>	1	1	0	0	5,155	2,709	1,885	126,508	278.86
61–75	<code>csearch</code>	30	19	0	11	103,095	31,404	37,261	26,516	143.70
41–50	<code>default</code>	120	120	0	0	748,952	0	191,780	12,665	37.01
51–60	<code>default</code>	1	1	0	0	24,685	0	8,797	661,951	1198.43
61–75	<code>default</code>	30	20	0	10	312,897	0	110,118	34,162	187.03

16,949 records.

Against published values: all 84 Small graphs agree with the pathwidths of [Mallach \(2018, Tables 4–5\)](#), and ten of the eleven VLSI circuits are certified at their published best-known track counts ([de Oliveira and Lorena, 2002, Table I](#)); the eleventh, W4 with 202 nets, is open at 28 against 27. One caution about MOSP benchmark files emerged: the same public repository ships the Carvalho and Soma instances as customers by patterns and the Chu and Stuckey instances beside them as patterns by customers, and only published densities and values tell them apart. Read the wrong way round, an instance certifies at a different value.

What is left is priced. One more budget step for every open class that fits our engine (1,024 vertices per component) costs at most 500 core-hours, and on the Rome graphs each such step has closed about 40% of what it attempted. The largest MOSP instances, 400 to 1,000 customers, and the 154 classes above 1,024 vertices are beyond an exact search. Refutation-certified records rest on the repaired search; for those within reach of the certificate emitter, the proof object above is the route by which a third party can check them without our code.

Table 5.4: The dataset by collection. *Classes*: isomorphism classes within the collection (nauty certificates; 78 graphs above 5,000 vertices by file content only). *Owned*: classes whose first member is in this collection, so the column sums to the dataset’s class count. The provenance columns count owned classes with a witness layout.

Collection	Files	Classes	Owned	Refutation	Bound	Upper bound	No witnessed value
MOSP: Challenge 2005	5,806	3,169	3,169	3,168	1	0	0
MOSP: Challenge 2005 (second copy)	46	46	0	0	0	0	0
MOSP: Chu & Stuckey	200	200	200	198	0	2	0
MOSP: Faggioli & Bentivoglio	300	287	274	274	0	0	0
MOSP: SCOOP	24	24	24	24	0	0	0
VLSI gate matrix circuits	11	11	11	5	5	1	0
VSPLIB trees	50	28	28	28	0	0	0
VSPLIB grids	50	50	50	9	0	19	22
VSPLIB Harwell-Boeing	73	72	72	32	7	33	0
Small (Marti et al. 2008)	84	84	84	32	52	0	0
Rome graphs	11,534	11,199	11,199	8,126	2,746	326	1
TreewidthLIB colouring	58	58	58	18	13	27	0
freetdi named graphs	150	146	143	97	22	23	1
freetdi control-flow graphs	1,817	1,070	1,064	398	661	4	1
MOSP: Frinhani et al. 2018, large	610	610	610	0	0	0	610
PACE 2016	291	288	112	37	18	51	6
PACE 2017 exact	200	199	193	61	7	115	10
PACE 2017 heuristic	200	197	173	11	3	45	114
PACE 2017 bonus	100	100	100	23	0	77	0
MOSP: Carvalho & Soma 2015	150	150	150	11	0	139	0
All	21,754		17,714	12,552	3,535	862	765

6 Conclusion

Placeholder. Section 6 is written last, once Sections 1–5 are final. It is to say what the complex is, what is proved and where, and what remains open: the missing Table 1 reference (Kashiwabara and Fujisawa 1979); the Table 1 claims that turned out false or only bands; LaPaugh’s $es = pes$, stated and not formalised; the three re-certifications still running; the certificate’s acceptance condition, argued on paper and not in Lean; and the gap between pathwidth and treewidth on trees of cliques, where no degree or clique bound can close it.

Data, code and proofs

Everything this paper reports can be checked from its repository, URL to be added on release. The repository holds:

- the Lean 4 development, with every theorem the paper names (Appendix A) and an axiom check that covers all of them;
- the source of both solvers, the open-stacks customer search and the graph pathwidth solver, in C and in Python, with the published and the repaired rules;
- the dataset of Section 5.2, the certificates of Section 5.1 and the independent checkers of both;
- the bibliometric data of Section 3: the cached OpenAlex responses, the relevance labels and the mechanical rule;
- a script for every table, figure and number in the paper, each of which regenerates it from the recorded data.

References

- J. Christopher Beck, Ryo Kuroiwa, Jimmy H. M. Lee, Peter J. Stuckey, and Allen Z. Zhong. Transition dominance in domain-independent dynamic programming. In *31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*, volume 340 of *LIPIcs*, pages 5:1–5:23, 2025. doi: 10.4230/LIPIcs.CP.2025.5.
- Daniel Bienstock and Paul Seymour. Monotonicity in graph searching. *Journal of Algorithms*, 12(2):239–245, 1991. doi: 10.1016/0196-6774(91)90003-H.
- Hans L. Bodlaender, Fedor V. Fomin, Arie M. C. A. Koster, Dieter Kratsch, and Dimitrios M. Thilikos. A note on exact algorithms for vertex ordering problems on graphs. *Theory of Computing Systems*, 50(3): 420–432, 2012. doi: 10.1007/s00224-011-9312-0.
- Geoffrey Chu and Peter J. Stuckey. Minimizing the maximum number of open stacks by customer search. In *Principles and Practice of Constraint Programming (CP 2009)*, volume 5732 of *Lecture Notes in Computer Science*, pages 242–257. Springer, 2009. doi: 10.1007/978-3-642-04244-7_21.
- Geoffrey G. Chu. *Improving Combinatorial Optimization*. PhD thesis, Department of Computer Science and Software Engineering, The University of Melbourne, 2011. Minerva Access, <http://hdl.handle.net/11343/36679>.
- David Coudert, Dorian Mazaauric, and Nicolas Nisse. Experimental evaluation of a branch and bound algorithm for computing pathwidth. In *Experimental Algorithms (SEA 2014)*, volume 8504 of *Lecture Notes in Computer Science*, pages 46–58. Springer, 2014. doi: 10.1007/978-3-319-07959-2_5.
- David Coudert, Dorian Mazaauric, and Nicolas Nisse. Experimental evaluation of a branch-and-bound algorithm for computing pathwidth and directed pathwidth. *ACM Journal of Experimental Algorithmics*, 21: 1–23, 2016. doi: 10.1145/2851494.
- Marco Antonio Moreira de Carvalho and Nei Yoshihiro Soma. A breadth-first search applied to the minimization of the open stacks. *Journal of the Operational Research Society*, 66(6):936–946, 2015. doi: 10.1057/jors.2014.60.
- A. C. M. de Oliveira and L. A. N. Lorena. A constructive genetic algorithm for gate matrix layout problems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 21(8):969–974, 2002. doi: 10.1109/TCAD.2002.800454.
- Holger Dell, Thore Husfeldt, Bart M. P. Jansen, Petteri Kaski, Christian Komusiewicz, and Frances A. Rosamond. The first parameterized algorithms and computational experiments challenge. In *11th International Symposium on Parameterized and Exact Computation (IPEC 2016)*, volume 63 of *LIPIcs*, pages 30:1–30:9, 2017. doi: 10.4230/LIPIcs.IPEC.2016.30.
- Holger Dell, Christian Komusiewicz, Nimrod Talmon, and Mathias Weller. The PACE 2017 parameterized algorithms and computational experiments challenge: The second iteration. In *12th International Symposium on Parameterized and Exact Computation (IPEC 2017)*, volume 89 of *LIPIcs*, pages 30:1–30:12, 2018. doi: 10.4230/LIPIcs.IPEC.2017.30.
- Giuseppe Di Battista, Ashim Garg, Giuseppe Liotta, Roberto Tamassia, Emanuele Tassinari, and Francesco Vargiu. An experimental comparison of four graph drawing algorithms. *Computational Geometry*, 7(5–6): 303–325, 1997. doi: 10.1016/S0925-7721(96)00005-3.

- Abraham Duarte, Laureano F. Escudero, Rafael Martí, Nenad Mladenovic, Juan José Pantrigo, and Jesús Sánchez-Oro. Variable neighborhood search for the vertex separation problem. *Computers & Operations Research*, 39(12):3247–3255, 2012. doi: 10.1016/j.cor.2012.04.017.
- J. A. Ellis, I. H. Sudborough, and J. S. Turner. The vertex separation and search number of a graph. *Information and Computation*, 113(1):50–79, 1994. doi: 10.1006/inco.1994.1064.
- Enrico Faggioli and Carlo Alberto Bentivoglio. Heuristic and exact methods for the cutting sequencing problem. *European Journal of Operational Research*, 110(3):564–575, 1998. doi: 10.1016/S0377-2217(97)00268-3.
- Michael R. Fellows and Michael A. Langston. Nonconstructive advances in polynomial-time complexity. *Information Processing Letters*, 26(3):157–162, 1987. doi: 10.1016/0020-0190(87)90054-8.
- Michael R. Fellows and Michael A. Langston. On search decision and the efficiency of polynomial-time algorithms. In *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing (STOC '89)*, pages 501–512, 1989. doi: 10.1145/73007.73055.
- Andreas Fink and Stefan Voß. Applications of modern heuristic search methods to pattern sequencing problems. *Computers & Operations Research*, 26(1):17–34, 1999. doi: 10.1016/S0305-0548(98)80001-4.
- Claudia Fink. *O problema de minimização de pilhas abertas – novas contribuições*. PhD thesis, Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2012.
- Fedor V. Fomin. Helicopter search problems, bandwidth and pathwidth. *Discrete Applied Mathematics*, 85(1):59–70, 1998. doi: 10.1016/S0166-218X(97)00131-5.
- Rafael de Magalhães Dias Frinhani, Marco Antonio Moreira de Carvalho, and Nei Yoshihiro Soma. A PageRank-based heuristic for the minimization of open stacks problem. *PLOS ONE*, 13(8):e0203076, 2018. doi: 10.1371/journal.pone.0203076.
- Maria Garcia de la Banda and Peter J. Stuckey. Dynamic programming to minimize the maximum number of open stacks. *INFORMS Journal on Computing*, 19(4):607–617, 2007. doi: 10.1287/ijoc.1060.0205.
- Toshinobu Kashiwabara and Toshio Fujisawa. NP-completeness of the problem of finding a minimum clique number interval graph containing a given graph as a subgraph. In *Proceedings of the 1979 IEEE International Symposium on Circuits and Systems*, pages 657–660, Tokyo, Japan, 1979.
- Nancy G. Kinnarsley. The vertex separation number of a graph equals its path-width. *Information Processing Letters*, 42(6):345–350, 1992. doi: 10.1016/0020-0190(92)90234-M.
- Lefteris M. Kirousis and Christos H. Papadimitriou. Interval graphs and searching. *Discrete Mathematics*, 55(2):181–184, 1985. doi: 10.1016/0012-365X(85)90046-9.
- Lefteris M. Kirousis and Christos H. Papadimitriou. Searching and pebbling. *Theoretical Computer Science*, 47:205–218, 1986. doi: 10.1016/0304-3975(86)90146-5.
- Kenta Kitsunai, Yasuaki Kobayashi, Keita Komuro, Hisao Tamaki, and Toshihiro Tano. Computing directed pathwidth in $O(1.89^n)$ time. *Algorithmica*, 75(1):138–157, 2016. doi: 10.1007/s00453-015-0015-9.

- Yasuaki Kobayashi, Keita Komuro, and Hisao Tamaki. Search space reduction through commitments in pathwidth computation: An experimental study. In *Experimental Algorithms (SEA 2014)*, volume 8504 of *Lecture Notes in Computer Science*, pages 388–399. Springer, 2014. doi: 10.1007/978-3-319-07959-2_33.
- András Kornai and Zsolt Tuza. Narrowness, pathwidth, and their application in natural language processing. *Discrete Applied Mathematics*, 36(1):87–92, 1992. doi: 10.1016/0166-218X(92)90208-R.
- Andrea S. LaPaugh. Recontamination does not help to search a graph. *Journal of the ACM*, 40(2):224–245, 1993. doi: 10.1145/151261.151263.
- L. Larisch and F. Salfelder. freetdi: named graphs and control-flow graphs. <https://github.com/freetdi/named-graphs>, <https://github.com/freetdi/CFGs>, n.d. Licence CC0 1.0.
- Thomas Lengauer. Black-white pebbles and graph separation. *Acta Informatica*, 16(4):465–475, 1981. doi: 10.1007/BF00264496.
- Alexandre Linhares. *Industrial Pattern Sequencing Problems: Some Complexity Results and New Local Search Models*. PhD thesis, Instituto Nacional de Pesquisas Espaciais (INPE), São José dos Campos, Brazil, 2001. Doctoral thesis in Applied Computing, advisor H. H. Yanasse.
- Alexandre Linhares and Horacio Hideki Yanasse. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers & Operations Research*, 29(14):1759–1772, 2002. doi: 10.1016/S0305-0548(01)00054-5.
- Sven Mallach. Linear ordering based MIP formulations for the vertex separation or pathwidth problem. *Journal of Discrete Algorithms*, 52–53:156–167, 2018. doi: 10.1016/j.jda.2018.11.012.
- Rafael Martí, Vicente Campos, and Estefanía Piñana. A branch and bound algorithm for the matrix bandwidth minimization. *European Journal of Operational Research*, 186(2):513–528, 2008. doi: 10.1016/j.ejor.2007.02.004.
- Mateus Martin, Horacio Hideki Yanasse, and Maria José Pinto. Mathematical models for the minimization of open stacks problem. *International Transactions in Operational Research*, 29(5):2944–2967, 2022. doi: 10.1111/itor.13053.
- Rolf H. Möhring. Graph problems related to gate matrix layout and PLA folding. In *Computational Graph Theory*, volume 7 of *Computing Supplementum*, pages 17–51. Springer-Verlag, Wien, 1990. doi: 10.1007/978-3-7091-9076-0_2.
- Tatsuo Ohtsuki, Hajimu Mori, Ernest S. Kuh, Toshinobu Kashiwabara, and Toshio Fujisawa. One-dimensional logic gate assignment and interval graphs. *IEEE Transactions on Circuits and Systems*, 26(9):675–684, 1979. doi: 10.1109/TCS.1979.1084695.
- Jason Priem, Heather Piwowar, and Richard Orr. OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv:2205.01833, 2022.
- Barbara M. Smith and Ian P. Gent. Constraint modelling challenge 2005. In *Fifth Workshop on Modelling and Solving Problems with Constraints, held at IJCAI 2005*, Edinburgh, 2005.
- Karol Suchan and Yngve Villanger. Computing pathwidth faster than 2^n . In *Parameterized and Exact Computation (IWPEC 2009)*, volume 5917 of *Lecture Notes in Computer Science*, pages 324–335. Springer, 2009. doi: 10.1007/978-3-642-11269-0_27.

- J.-W. van den Broek and Hans L. Bodlaender. TreewidthLIB. <http://www.cs.uu.nl/research/projects/treewidthlib/>, n.d. Graph files no longer served; colouring subset obtained from <https://mat.tepper.cmu.edu/COLOR/instances.html>.
- Omar Wing, S. Huang, and R. Wang. Gate matrix layout. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 4(3):220–231, 1985. doi: 10.1109/TCAD.1985.1270118.
- Horacio Hideki Yanasse. A transformation for solving a pattern sequencing problem in the wood cut industry. *Pesquisa Operacional*, 17(1):57–70, 1997a.
- Horacio Hideki Yanasse. On a pattern sequencing problem to minimize the maximum number of open stacks. *European Journal of Operational Research*, 100(3):454–463, 1997b. doi: 10.1016/S0377-2217(97)84107-0.
- Horacio Hideki Yanasse and Edson Luiz França Senne. The minimization of open stacks problem: A review of some properties and their use in pre-processing operations. *European Journal of Operational Research*, 203(3):559–567, 2010. doi: 10.1016/j.ejor.2009.09.017.

A Formal names

Every theorem, lemma and counterexample of this paper is checked in the Lean 4 proof assistant with the Mathlib library, unless the text says otherwise. The reader need not read Lean to follow the paper. This appendix is for a reader who wants to find the formal statement of a result: Table A.1 maps each result, in the order in which it appears, to the names of the Lean declarations that state it. Where a row cites a definition rather than a theorem, it is the definition the text uses.

All the theorems named depend only on the three standard axioms of Lean’s classical logic, `propext`, `Classical.choice` and `Quot.sound`; an axiom check in the repository prints this for every theorem named in the paper. The development has one `sorry`, a conjecture about lower bounds that no result of this paper uses. The part of the development behind Section 4 has no `sorry` at all.

One Lean proof takes a different route from the published one: the proof that node search is monotone (Theorem 2.4(5)) follows the crusade argument of Bienstock and Seymour (1991), not the reduction to edge search of LaPaugh (1993) on which Kirousis and Papadimitriou (1986) rely. A few facts in the paper are computed rather than proved: that every end of the two bands occurs (the values of K_2 , $K_{1,3}$ and $K_{3,3}$ after Theorem 2.5), each computed by brute force from the quantity’s own definition; the search for smaller counterexamples (Appendix B.5); and the computational results of Section 5.

Table A.1: Each result of the paper and its formal statement in Lean.

Entry	Result in the paper	Lean names
<i>Section 2, the equivalences</i>		
A.1	Lemma 2.1, cliques sit in a bag	<code>PathDecomposition.exists_bag_of_isClique</code>
A.2	Theorem 2.2, vertex separation is pathwidth	<code>vertexSeparation_eq_pathwidth</code>
A.3	Theorem 2.3, MOSP is pathwidth plus one	<code>MOSPInstance.mospValue_eq_pathwidth_add_one</code>
A.4	The pattern graph gives no bound: the star $K_{1,4}$ (after Theorem 2.3)	<code>star_mospValue,</code> <code>star_pathwidth_agreementGraph,</code> <code>star_refutes_pattern_graph_bound</code>

Entry	Result in the paper	Lean names
A.5	Theorem 2.4(1), gate matrix layout	NetGateMatrix.tracks_eq_mospValue, NetGateMatrix.tracks_eq_pathwidth_add_
A.6	Theorem 2.4(2), interval thickness	one intervalThickness_eq_pathwidth_add_
A.7	Theorem 2.4(3), one-dimensional logic	one LogicArray.tracks_eq_
A.8	Theorem 2.4(4), narrowness	intervalThickness, LogicArray.tracks_
A.9	Theorem 2.4(5), node search	eq_pathwidth_add_one narrowness_eq_pathwidth_add_one
A.10	Theorem 2.4(6), multiple folding	nodeSearchMonotonicity, nodeSearch_eq_
A.11	Theorem 2.4(7), Lengauer's vertex separator game	vertexSeparation_add_one, nodeSearch_chain
A.12	Theorem 2.5, the bands	NetGateMatrix.foldTracks_eq_pathwidth_
A.13	Theorem 2.6(1), PLA folding	add_one vsg_eq_max
A.14	Theorem 2.6(2), edge separation read as cutwidth	pathwidth_le_splitBandwidth_le_
<i>Section 4, the search</i>		pathwidth_add_one, vertexSeparation_le_
A.15	Step cost, P_k and Sol_k (Section 4.1)	edgeSearch_le_add_two, vertexSeparation_le_
A.16	Lemma 4.1, free moves never hurt	progressiveEdgeSearch_le_add_two
A.17	Theorem 4.2, the search decides pathwidth	plaTracks_idMatrix_five, plaTracks_idMatrix_unbounded,
A.18	Node-soundness of a filter (Section 4.1)	plaTracks_pathMatrix_unbounded
A.19	Lemma 4.3, what the premise says	cutwidth_unbounded, modCutwidth_unbounded
A.20	Counterexample 4.4	stepCost, Solvable, SearchSol
A.21	Chu and Stuckey's Theorem 1, under both readings, and Chu's Theorem 6.3.6 are false	solvable_iff_searchSol_cl
A.22	Fink's restatement of the definite move is false	searchSol_empty_iff_pathwidth_add_one_
A.23	The hereditarily definite premise, and its failure in Counterexample 4.4	le, searchSol_mospGraph_iff_mospValue_le
A.24	Lemma 4.5, open stacks are submodular	NodeSoundAt
A.25	Theorem 4.6, the repaired definite move is sound	closeCount_eq, isDefinite_iff
A.26	Theorem 4.7, the repair is a matching condition	definiteMove_counterexample, cexFamily_closed
A.27	The published definite move is right when $\text{open}(q, S) \leq 1$	chuStuckey_theorem1_false, chuStuckey_theorem1_false_literal,
A.28	The subset rule is sound	chuThesis_theorem636_false
		fink_theorem1_false
		IsHereditarilyDefinite,
		not_isHereditarilyDefinite_cex
		openStacks_submodular
		solvable_cl_insert_of_
		hereditarilyDefinite, searchSol_cl_
		insert_of_hereditarilyDefinite
		isHereditarilyDefinite_iff_
		hasDefiniteMatching
		isHereditarilyDefinite_of_openCount_
		le_one
		searchSol_cl_insert_of_newlyOpened_
		subset, subsetFilter_sound

Entry	Result in the paper	Lean names
A.29	The subset rule needs its tie-break (7-customer graph)	noTieBreak_counterexample
A.30	The better move's premise, restated	isBetter_iff
A.31	Chu and Stuckey's Theorem 2 (the better move) is false	betterMove_counterexample, chuStuckey_theorem2_false
A.32	Chu's Theorem 6.3.8 is false	chuThesis_theorem638_false, chuThesis_theorem638_false_literal
A.33	The repaired better move is sound	IsRepairedBetter, searchSol_cl_insert_of_repairedBetter
A.34	Two implementation faults of the better move	bugA_counterexample, bugB_counterexample
A.35	The old move is sound	searchSol_reinsert, not_searchSol_of_oldMove
A.36	The old move needs its reinsertion test (path on five vertices)	reinsert_needs_test
A.37	Solvability of a state does not depend on the path to it (the memo)	solvable_iff_of_cl_eq
A.38	A node with an unrefuted old move can record a solvable state	exec_fake_oldMove
A.39	Theorem 4.8, the revised search is sound	exec_repairedFullFilter_sound, exec_repairedFullFilter_pathwidth, exec_repairedFullFilter_mospValue
A.40	Under the published rules the filter is not node-sound	not_codeFilterSound_cexGraph
A.41	Lemma 4.9, the premises as commitment conditions	isDefinite_iff_endpoint, isHereditarilyDefinite_iff_ isCommittable
A.42	In Counterexample 4.4 the endpoint test passes and the commitment fails	cex_isDefinite_not_isCommittable
A.43	The published premise gives a commitment to the least-border set	exists_isCommittable_of_isDefinite
A.44	In Counterexample 4.4 that set is $\{2, 3, 4\}$	cex_isCommittable_234
A.45	A commitment of depth one holds iff $\text{open}(q, S) \leq 1$	isCommittable_insert_iff
A.46	The greedy step of Coudert, Mazauric and Nisse is that condition	isGreedyStep_iff
<i>Table 4.1, the dictionary</i>		
A.47	Table 4.1: closing order from $\emptyset$	orderOfLayout
A.48	Table 4.1: open stacks $b(T)$	openStacks_eq_card_boundary
A.49	Table 4.1: $\text{cost}(T, c)$	stepCost_eq_card_boundary_add_one
A.50	Table 4.1: $\text{Sol}_k(\emptyset)$	searchSol_empty_iff_pathwidth_add_one_ le
A.51	Table 4.1: free closure $\text{cl}(T)$	mem_cl_iff
A.52	Table 4.1: definite move, repaired	isHereditarilyDefinite_iff_ isCommittable
<i>Section 5, computational results</i>		
A.53	The split of the root into independent tasks is sound	execSplit_repairedFullFilter_sound, execSplit_repairedFullFilter_ mospValue
<i>Appendix B, supporting material</i>		

Entry	Result in the paper	Lean names
A.54	Node search equalities need an edge (Appendix B.2)	nodeSearch_ne_intervalThickness_of_edgeless, monotoneNodeSearch_ne_vertexSeparation_add_one_of_edgeless
A.55	Lengauer’s Theorem 4 fails at $K = 0$ (Appendix B.2)	isPositiveVSG_triangleGraph_counterexample
A.56	$t = \text{ns}$ fails on the matrix [1] (Appendix B.2)	tracks_ne_monotoneNodeSearch_one
A.57	The Kirousis–Papadimitriou claim fails on $K_{1,3}$ (Appendix B.2)	kirousisPapadimitriou_claim2_false
A.58	Progressive pebbling is in the exact core (Appendix B.3)	pbw_lengauerD_eq_pathwidth, pbw_eq_mospValue_pebbleMatrix, mpb_eq_pathwidth_add_one
A.59	Proposition B.1, the repaired filter is node-sound	repairedFullFilter_sound
A.60	Theorem B.2, runs are sound	Exec.sound

B Supporting material

This appendix holds the material that supports Sections 2 and 4 without being needed to follow them.

B.1 Formal definitions of eight problems

These are the formal definitions of the eight problems that Table 2.1 states in one line. As in Section 2.1, each is a decision problem with an integer k , and its value is the least k for which the answer is yes.

One-dimensional logic. Given gates, nets and the nets each gate connects, is there an order of the gates whose net intervals have an interval graph of clique number at most k ? Its *net graph* H joins two nets that share a gate. A *boundary* variant pins two gates to the ends (Ohtsuki et al., 1979, §§II–IV).

PLA folding $\text{pla}(M)$. As gate matrix layout, with at most two nets per track. With no such bound (*multiple folding*) it is gate matrix layout itself (Möhring, 1990, pp. 25, 36).

Interval thickness $\theta(G)$. Is there an interval supergraph of G on the same vertices with clique number at most k ? (Kirousis and Papadimitriou, 1985, p. 182); (Möhring, 1990, p. 31).

Node search $\text{ns}(G)$. Can searchers, placed on and removed from vertices, clear every edge with at most k on the graph at once, where an edge is cleared when both ends are guarded and a clear edge is recontaminated by a searcher-free path to a contaminated one? $\text{mns}(G)$ is the same over strategies that never recontaminate (Kirousis and Papadimitriou, 1985, p. 181); (Kirousis and Papadimitriou, 1986, §2).

Edge search $\text{es}(G)$. The same game with a third move, sliding a searcher along an edge, which clears that edge; $\text{pes}(G)$ for progressive strategies, which never recontaminate an edge (Kirousis and Papadimitriou, 1986, p. 208); (Ellis et al., 1994, p. 53).

Narrowness $\nu(G)$. Is there an order $v_1, \dots, v_n$ such that the following process never holds more than k vertices: at step i add v_i , then remove every v_j , $j \leq i$, with no neighbour v_ℓ , $\ell > i$? (Kornai and Tuza, 1992, §2).

Split bandwidth $\text{sb}(G)$. Is there a graph obtained from G by repeatedly splitting a vertex into an edge, its neighbours partitioned between the two ends, with bandwidth at most k ? (Fomin, 1998, §3.2).

Edge separation. The Linhares–Yanasse table cites [Lengauer \(1981\)](#), which supports three readings: cutwidth $\text{cw}(G)$, the least over layouts of the most edges crossing a gap (p. 468); modified cutwidth $\text{mcw}(G)$, counting only edges that pass strictly over a vertex (Definition 6); and Lengauer’s vertex separator game $\text{VSG}(G)$, the least K such that the vertices can be pebbled in an order that leaves at most K unpebbled vertices adjacent to pebbled ones (p. 467).

B.2 Edge cases and a gap in a published proof

Three of the published equalities fail on degenerate inputs, and each failure is proved in Lean. On a nonempty graph with no edges, $\text{ns} = 0$ while $\text{vs} + 1 = \theta = 1$, so both equalities of [Kirosis and Papadimitriou \(1985, 1986\)](#) need an edge (Appendix A, A.54). For a matrix with no 1s, $Z = 0$ while $\text{pw} + 1 = 1$. Lengauer’s Theorem 4 fails at $K = 0$ on an edgeless graph (Appendix A, A.55). Möhring (1990, Thm 3.9), $t = \text{ns}$, fails literally on the 1×1 matrix [1] (Appendix A, A.56).

The published proof of $\text{vs} \leq \text{ns} - 1$ ([Kirosis and Papadimitriou, 1986](#), Thm 4.1) has a gap. It orders the vertices by when they first receive a searcher and claims that every vertex counted at the worst cut still holds a searcher, “since the vertices in D_{i_0} are joined to vertices not yet cleared” (p. 217). A strategy may place a searcher and remove it again before clearing anything, which recontaminates nothing. On $K_{1,3}$, placing and removing a searcher on each leaf before guarding the centre is an optimal recontamination-free strategy for which the claim fails (Appendix A, A.57). The theorem is true. The repair orders the vertices by clearing time, as our Lean proof does, or uses the paper’s own normal form, its Corollary 2.4.

B.3 A thirteenth member

Progressive black-white pebbling, which is not in the Linhares–Yanasse table, belongs to the exact core. [Lengauer \(1981, Thm 3\)](#) gives $\text{vs}(G) \leq K \iff \text{pbw}(G_d) \leq K + 2$ on the dag G_d built from G ; his Theorem 2 gives $\text{pbw}(D) = \text{vs}(D_u) + 1$ on a dag D , where D_u is the MOSP graph of the matrix whose column v is v with its predecessors; and the minimum progressive pebbling number of [Kirosis and Papadimitriou \(1986, Thm 3.1\)](#) equals $\text{pw} + 1$. All three are proved in Lean, each with its hypothesis: Lengauer’s Theorem 2 in its instance form fails at $K = 1$ on a dag with no arcs, and $\text{mpb} = \text{ns}$ fails on edgeless graphs (Appendix A, A.58).

B.4 Why the published proof of the definite move fails

The proof moves q to the front of a solution and claims that after each prefix “we have at most $\text{open}(q, S)$ extra stacks open, but at least $\text{close}(q, S)$ extra stacks closed” (p. 6). At a prefix $T \supseteq S$ that does not yet contain q , moving q forward changes the open stacks by $b(T \cup X) - b(T) = |N[q] \setminus O(T)| - |X \setminus T|$. At $T = S$ this is $\text{open} - \text{close} \leq 0$. But customers of X that the solution has *already* closed are in T and no longer count as extra stacks closed, while the stacks q opens may still be new. In the example, a solution may begin with 3, after which 4 is free: at $T = \{2, 3, 4\}$, moving 0 forward costs one more stack. A second, independent slip: even where the conclusion holds, the literal sequence $S ++ [q] ++ R'$ need not be a solution, because a customer that q frees stays open until its turn; the argument needs the free moves.

B.5 The search for smaller counterexamples

The smallest graph on which the rule loses the last solution has 14 customers: there is none on any graph to 9 vertices, by exhaustion, and none on the 88,802 connected graphs on 10 vertices with at most 14 edges. The counterexample was found by a search over a family of gadgets built from the failed proof, and minimised by deletion.

B.6 The filter and the runs are sound

Proposition B.1. *At every state with the invariant, for every set Q of refuted old moves, the filter definite $\rightarrow$ subset $\rightarrow$ better with both repairs is node-sound (Appendix A, A.59).*

Proof. If the definite move fires, Theorem 4.6 applies. Otherwise the subset rule keeps a survivor with a solution; let w be the earliest. If the better move dropped w , it cited an earlier survivor that the repaired argument proves solvable, contradicting the choice of w . $\square$

Theorem B.2 (runs are sound). *Let the filter be node-sound and return only playable customers. If at the start of a run the invariant holds, every memo entry is a genuine refutation, and every old move is refuted, then the run leaves only genuine memo entries, and if it answers false at S then $\neg \text{Sol}_k(S)$. This holds for any child order and any memo policy, with the old move on or off (Appendix A, A.60).*

The proof is an induction over the run, in the order in which subtrees complete: every old move a node holds is a sibling subtree that completed with *false* earlier in the same run, or an inherited one that passed the reinsertion test.

C The bibliometric study

This appendix gives the method behind Section 3 in full, the counts per name and when each name was in use.

Search. We measured how much each name is used with OpenAlex (Priem et al., 2022), on 29 September 2026. For each of the twelve problems we searched titles and abstracts for its name and its common variants, and restricted the search to the four fields in which the problems are studied: Computer Science, Mathematics, Engineering and Decision Sciences. Without that restriction several names are ordinary words: “narrowness” returns over a million works, and “edge separation” returns aerodynamics.

Relevance. Inside those fields a search hit is still not a work that uses the name. OpenAlex’s search stems words and ignores punctuation, so a quoted phrase matches more than the phrase. “Narrowness” matches “narrowing” in term rewriting; “node searching” matches look-ups in peer-to-peer networks; “split bandwidth” matches radar interferometry and 5G spectrum assignment; “path-width” matches the widths of roads, tool paths and datapaths. We therefore fetched all 2,271 hits with their titles, abstracts, topics and venues, and kept a work only if it is about the problem in the sense of the Linhares–Yanasse table. For the eleven smaller names the 700 hits were judged one by one. For pathwidth, the 1,571 hits were filtered by a rule: keep a work that uses the one-word spelling, that OpenAlex places in a theory subfield, or that mentions graphs, treewidth or minors and not roads, vehicles, trajectories or datapaths. We checked the rule by reading samples of what it kept and what it dropped. Of the 2,271 hits, 1,593 are relevant: 1,213 for pathwidth and 380 for the other eleven names. The two search-game names keep 27% and 40% of their hits.

Robustness. The counts are lower bounds. A work can study a problem without naming it in its abstract, OpenAlex has few abstracts before about 1990, and its citation counts run below Google Scholar’s. The judgements were made by one annotator with model assistance, record a yes or no per work and not the reason, and have not been checked by a second annotator. As a second reading we applied a mechanical rule: the literal phrase with a context term and no exclusion term. It agrees with the judgements on 543 of the 700 judged works (78%). Under both readings pathwidth has more than three times as many works as the other eleven names together (3.2 and 3.6 times), the three empty names stay empty apart from one 5G

Table C.1: Works that use each problem’s name in the sense of the Linhares–Yanasse table, and citations of the paper the table cites for it. *Relevant*: hits kept by the relevance check. *Citations*: OpenAlex `cited_by_count`, 29 September 2026. Kashiwabara and Fujisawa (1979) has no OpenAlex record. “OpenAlex returned one pathwidth record twice: the counts are of records, as Figure 3.1 draws them, and there are 1,212 distinct relevant pathwidth works.

Problem	Discipline	Relevant	Hits	Reference in the 2002 table	Citations
Pathwidth	graph theory	1,213 ^a	1,571	Kinnersley (1992)	214
Gate matrix layout	VLSI	110	125	Möhring (1990) ; Wing et al. (1985)	134 / 89
PLA folding	VLSI	68	74	Möhring (1990)	134
MOSP	operations research	58	60	Yanasse (1997b) ; Fink and Voß (1999)	75 / 71
Vertex separation	graph theory	55	100	Kinnersley (1992)	214
Edge search	graph theory	38	95	Kirkousis and Papadimitriou (1986)	293
Node search	graph theory	34	127	Kirkousis and Papadimitriou (1985)	124
One-dimensional logic	VLSI	11	18	Ohtsuki et al. (1979)	108
Interval thickness	graph theory	6	10	Kashiwabara and Fujisawa (1979)	–
Narrowness	graph theory	0	35	Kornai and Tuza (1992)	44
Split bandwidth	graph theory	0	35	Fomin (1998)	19
Edge separation	graph theory	0	21	Lengauer (1981)	77
All twelve		1,593	2,271		

false positive, and the search games lose most of their hits. MOSP’s rank over all years is the one conclusion that depends on the reading: fourth under the judgements, fifth under the rule.

Counts and citations. Table C.1 gives the counts per name. Citation counts do not track name usage. [Kirkousis and Papadimitriou \(1986\)](#) is the most-cited paper in the table (293 citations), while only 38 works use “edge search” in its sense: the paper is cited as a foundational result on graph searching, not because people work on the edge search game under that name. The split is starker for [Ohtsuki et al. \(1979\)](#): 108 citations against 11 works using “one-dimensional logic”. Split bandwidth is Fomin’s own term in the paper the table cites ([Fomin, 1998](#)), and no one adopted it.

When each name was in use. Figure C.1 shows the counts over time. PLA folding and gate matrix layout peak in 1980–1989 and fall to almost nothing after 2010, and one-dimensional logic has no relevant work after 2009. MOSP first appears in the 1990s, peaks in 2005–2014 and is still active. Pathwidth’s rate has roughly doubled since the 1990s, from about 15 to 27 works per million, and its 311 relevant works in 2020–2024 are more than the whole history of any other name. Rows with a few works per period move by whole cells on one paper, and should be read as anecdote rather than trend.

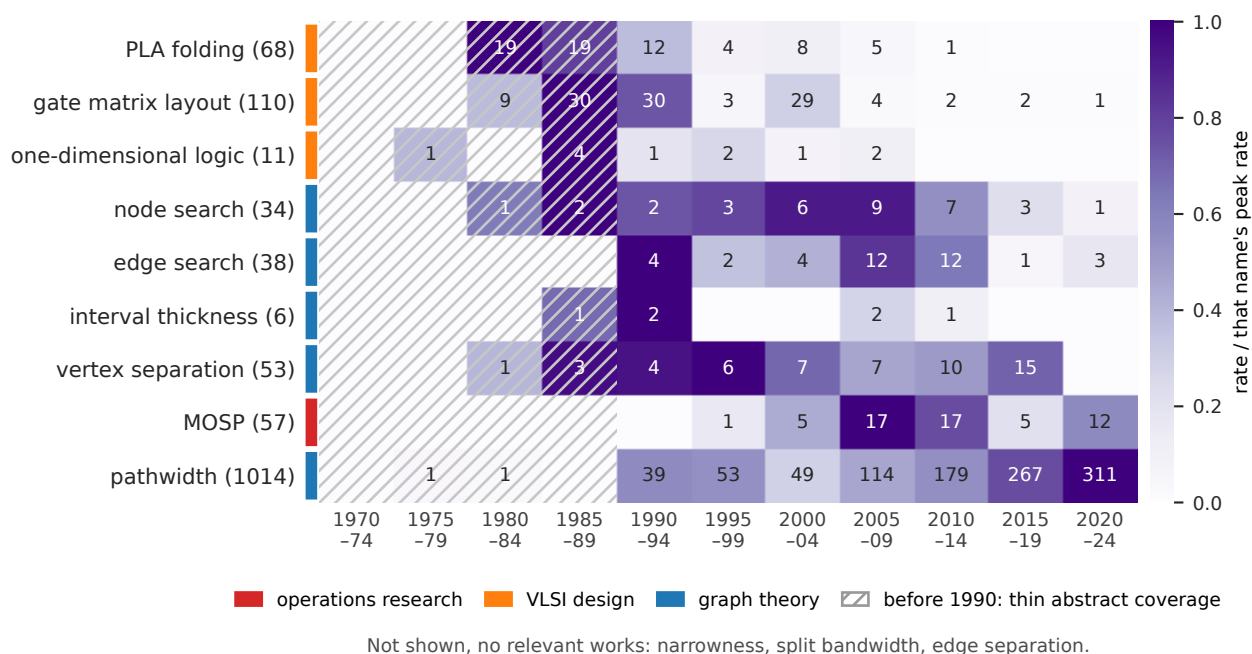


Figure C.1: The VLSI names belong to the 1980s, the search games peak later and fade, MOSP peaks in 2005–2014, and only pathwidth is still rising. The shade is the name’s rate, relevant works per million works in the same four fields in each five-year period, as a fraction of that name’s own peak rate; the numbers are raw counts. Rows are ordered by the period of their peak. Counts cover 1970–2024, since 2025–2026 is not fully indexed, so row totals are below those of Table C.1. Before 1990 (hatched) OpenAlex has few abstracts.